

Singleton Design Pattern

Senior/Staff Interview Handbook — Saudi Arabia, Dubai/UAE, Malaysia, India
Tier-2, India Tier-1/60LPA+ (Bilingual English + Hindi)

Interview Prep Handbook · Updated August 2026

Singleton Design Pattern

Fast Track (Parts 1–6) → Deep Dive (Parts 7–21) → Appendix (Part 22). Companion code: `Singleton.php` — all runnable examples referenced by name live there, not inlined here (code file is English-only).

फ़ास्ट ट्रैक (भाग 1–6) → डीप डाइव (भाग 7–21) → परिशिष्ट (भाग 22)। साथी कोड फ़ाइल: `Singleton.php` — नाम से रेफ़र किए गए सारे रनेबल उदाहरण वहीं हैं, यहाँ इनलाइन नहीं (कोड फ़ाइल सिर्फ़ अंग्रेज़ी में है)।

⚡ FAST TRACK

Part 1 — 60-Second Recall Card

One-liner	Singleton ensures a class has exactly one instance for the lifetime of the process, and provides one global access point to it.
GoF category	Creational
Core mechanism	Private (or protected) constructor + a private static instance property + a public static <code>getInstance()</code> method that lazily creates the instance on first call and returns the same cached object on every call after. Block <code>__clone()</code> and <code>__wakeup()</code> / <code>__unserialize()</code> so copying or deserializing can't produce a second instance.

Trigger phrase	"There must only ever be one X in the whole application" — one logger, one config manager, one connection pool, one cache manager, one ID generator.
Anti-trigger	You need more than one configured instance of the same class (→ Factory/Registry); you need to unit-test the consumer in isolation and mock this dependency (classic Singleton fights you here — prefer container-managed single-instance injection); "one per tenant/request/user" is not actually "one," it's scoped state wearing a Singleton costume.
Closest confused patterns	Static class (no instance exists at all — Singleton <i>is</i> an object, can implement interfaces, can be polymorphic, a static class cannot); Registry (many named instances behind string/enum keys — Singleton is exactly one, anonymous); Multiton (a keyed map of Singletons — "one per key," not "one, period"); DI-container "singleton scope" (what most engineers actually mean day-to-day — a container caches and reuses one instance per binding, but there's no private constructor and no <code>getInstance()</code> ; it's the same <i>idea</i> , deliberately without the GoF pattern's testability costs).
Memory hook	A country has exactly one sitting president at a time — the <i>office</i> enforces "only one," not the person's own willpower. Singleton is the class enforcing "only one instance of me" on itself, the same way.

हिंदी अनुवाद / Hindi Translation:

एक-पंक्ति सार	Singleton सुनिश्चित करता है कि एक क्लास के पास प्रोसेस के पूरे जीवनकाल के लिए ठीक एक इंस्टेंस हो, और उसे एक वैश्विक एक्सेस बिंदु प्रदान करता है।
GoF श्रेणी	क्रिएशनल (Creational)
मुख्य तंत्र	एक प्राइवेट (या प्रोटेक्टेड) कंस्ट्रक्टर + एक प्राइवेट स्टैटिक इंस्टेंस प्रॉपर्टी + एक पब्लिक स्टैटिक <code>getInstance()</code> मेथड जो पहली कॉल पर आलसी (lazily) रूप से इंस्टेंस बनाती है और हर बाद की कॉल पर वही कैश ऑब्जेक्ट लौटाती है। <code>__clone()</code> और <code>__wakeup()</code> / <code>__unserialize()</code> को ब्लॉक करें ताकि कॉपी करना या डीसीरियलाइज़ करना दूसरा इंस्टेंस न बना सके।
ट्रिगर वाक्यांश	"पूरे ऐप्लिकेशन में हमेशा सिर्फ एक X होना चाहिए" — एक logger, एक config manager, एक connection pool, एक cache manager, एक ID generator।
एंटी-ट्रिगर	आपको एक ही क्लास के एक से ज़्यादा कॉन्फ़िगर्ड इंस्टेंस चाहिए (→ Factory/Registry); आपको कंज़्यूमर को अलग-थलग यूनिट-टेस्ट करना है और इस डिपेंडेंसी को मॉक करना है (क्लासिक Singleton यहाँ आपसे लड़ता है — कंटेनर-प्रबंधित सिंगल-इंस्टेंस इंजेक्शन को प्राथमिकता दें); "प्रति-टेनेंट/रिक्वेस्ट/यूज़र एक" असल में "एक" नहीं है, यह Singleton का लबादा पहने स्कोप स्टेट है।
सबसे मिलते-जुलते भ्रामक पैटर्न्स	स्टैटिक क्लास (कोई इंस्टेंस बिल्कुल नहीं होता — Singleton एक ऑब्जेक्ट है, इंटरफ़ेसज़ इम्प्लीमेंट कर सकता है, पॉलीमॉर्फिक हो सकता है, एक स्टैटिक क्लास नहीं); Registry (स्ट्रिंग/enum की के पीछे कई नामित इंस्टेंसेज़ — Singleton ठीक एक है, अनाम); Multiton (Singletons का एक keyed मैप — "प्रति-की एक," "बस एक" नहीं); DI-कंटेनर "singleton scope" (जो ज़्यादातर इंजीनियर असल में रोज़ाना मतलब रखते हैं — एक कंटेनर हर बाइंडिंग के लिए एक इंस्टेंस कैश और पुनः इस्तेमाल करता है, लेकिन कोई प्राइवेट कंस्ट्रक्टर और कोई <code>getInstance()</code> नहीं है; यह वही विचार है, जानबूझकर GoF पैटर्न की टेस्टेबिलिटी लागतों के बिना)।
याद रखने की तरकीब	एक देश में एक समय में ठीक एक ही राष्ट्रपति होता है — पद "सिर्फ एक" लागू करता है, व्यक्ति की अपनी इच्छाशक्ति नहीं। Singleton वह क्लास है जो खुद पर "मेरा सिर्फ एक इंस्टेंस" उसी तरह लागू करती है।

Part 2 — Market Calibration

Sourced directly from `design-patterns-frequency-guide-expanded.md`. Singleton ranks #3 overall on the master frequency table, labeled *Very High*, with real-world anchors: *Logger*, *Config Manager*, *DB Connection Pool*.

सीधे `design-patterns-frequency-guide-expanded.md` से लिया गया। मास्टर फ्रीक्वेंसी टेबल पर *Singleton* कुल मिलाकर #3 पर है, जिसे *Very High* का दर्जा दिया गया है, असली-दुनिया के एंकर्स के साथ: *Logger*, *Config Manager*, *DB Connection Pool*।

Market	Singleton's standing	Evidence	What that means for prep
Malaysia	Headline pattern — the strongest single-pattern signal in any market in this guide.	Market summary states Singleton is "uniquely explicit" — candidates at both Maybank and AirAsia/Capital A report being asked to <i>write it out live</i> , with double-checked locking explicitly discussed at Maybank . Also named at Shopee and IBM Malaysia .	If you're interviewing in Malaysia, this is not optional depth — you need to write a thread-safe Singleton from memory, unprompted, and be ready to defend the thread-safety mechanism under follow-up. This is the one market/pattern combination in the whole guide with a named, repeated "write it live" signal.
India Tier-1 / 60LPA+	Strong #2, close behind Strategy.	Named at Uber India (alongside multithreading), Directi/Media.net , Oracle India , Mastercard India , and inferred at Grab (India-facing).	Expect it as one of 2–4 patterns a bar-raiser-caliber interviewer wants named and justified in a larger design, often alongside a concurrency or scale-up follow-up — not usually the sole focus of a round, but a near-certain supporting player.
India Tier-2	Top-4 recurring pattern , after Strategy.	Named at Razorpay , Postman (explicitly justified, not just implemented), ShareChat , Ola , Infosys , Cognizant , TCS Digital .	Very likely to come up in a machine-coding round for a <i>Logger/Config/Connection-Pool</i> -shaped component; interviewers here are more likely to ask you to say the pattern name out loud and justify the choice than to probe deep internals.
Saudi Arabia	Thin, single data point.	Only one company row names it: Accenture (KSA centers) , where Singleton is flagged "most important" alongside <i>Factory/Strategy/Observer</i> , at global (not country-specific) confidence.	Don't over-invest specifically for Saudi Arabia on this pattern beyond the baseline — the guide's honest position is that the country-specific evidence here is thin, not that the pattern is unimportant globally.
Dubai/ UAE		The UAE section's "most-asked pattern" line names Strategy,	Treat this as a genuine data gap, not a signal that Singleton is unused in

Market	Singleton's standing	Evidence	What that means for prep
	No company-level evidence in the guide.	Factory, and SAGA/pub-sub — Singleton does not appear in any of the 29 UAE company rows checked.	UAE interviews — it almost certainly still appears as a supporting pattern (it's foundational enough that most LLD rubrics touch it), but there's no direct citation to calibrate depth against. Prepare it at baseline depth rather than Malaysia-level intensity for UAE specifically.

हिंदी अनुवाद / Hindi Translation (टेबल का सार, बाज़ार दर हिसाब से):

Malaysia — हेडलाइन पैटर्न, इस पूरी गाइड में किसी भी बाज़ार में सबसे मज़बूत अकेला-पैटर्न संकेत। बाज़ार सार कहता है कि Singleton "विशिष्ट रूप से स्पष्ट" है — Maybank और AirAsia/Capital A दोनों में उम्मीदवारों ने बताया कि उन्हें इसे लाइव लिखने को कहा गया, Maybank में double-checked locking की स्पष्ट चर्चा के साथ। Shopee और IBM Malaysia पर भी नामित। तैयारी के लिए मतलब: अगर आप Malaysia में इंटरव्यू दे रहे हैं, यह वैकल्पिक गहराई नहीं है — आपको बिना पूछे याद से एक थ्रेड-सेफ़ Singleton लिखना होगा, और फ़ॉलो-अप में थ्रेड-सेफ़्टी तंत्र का बचाव करने के लिए तैयार रहना होगा।

India Tier-1 / 60LPA+ — Strategy के ठीक पीछे एक मज़बूत #2। Uber India (मल्टीथ्रेडिंग के साथ), Directi/Media.net, Oracle India, Mastercard India पर नामित, Grab पर अनुमानित। तैयारी के लिए मतलब: एक बड़े डिज़ाइन में एक बार-रेज़र-स्तर इंटरव्यूअर द्वारा नामित और सही ठहराए जाने वाले 2-4 पैटर्न्स में से एक की उम्मीद रखें, अक्सर एक concurrency या स्केल-अप फ़ॉलो-अप के साथ।

India Tier-2 — Strategy के बाद टॉप-4 दोहराया जाने वाला पैटर्न। Razorpay, Postman (खुलकर सही ठहराया गया, सिर्फ़ इम्प्लीमेंट नहीं), ShareChat, Ola, Infosys, Cognizant, TCS Digital पर नामित। तैयारी के लिए मतलब: एक Logger/Config/Connection-Pool-आकार के कंपोनेंट के लिए मशीन-कोडिंग राउंड में आने की बहुत संभावना; यहाँ इंटरव्यूअर्स गहरी आंतरिक कार्यप्रणाली जाँचने से ज़्यादा आपसे पैटर्न का नाम ज़ोर से बताने और चुनाव सही ठहराने को कह सकते हैं।

Saudi Arabia — पतला, अकेला डेटा बिंदु। सिर्फ़ एक कंपनी-पंक्ति इसका नाम लेती है: Accenture (KSA केंद्र), जहाँ Singleton को Factory/Strategy/Observer के साथ "सबसे महत्वपूर्ण" चिह्नित किया गया है, वैश्विक (देश-विशिष्ट नहीं) भरोसे पर। तैयारी के लिए मतलब: इस पैटर्न पर Saudi Arabia के लिए बुनियादी स्तर से ज़्यादा अतिरिक्त निवेश न करें।

Dubai/UAE — गाइड में कोई कंपनी-स्तर सबूत नहीं। UAE सेक्शन की "सबसे ज़्यादा पूछा गया पैटर्न" पंक्ति Strategy, Factory, और SAGA/pub-sub का नाम लेती है — Singleton जाँची गई 29 UAE कंपनी-पंक्तियों में से किसी में नहीं दिखता। तैयारी के लिए मतलब: इसे एक असली डेटा गैप के तौर पर लें, यह संकेत नहीं कि UAE इंटरव्यूज़ में Singleton इस्तेमाल नहीं होता — इसे UAE के लिए Malaysia-स्तर की तीव्रता के बजाय बुनियादी गहराई पर तैयार करें।

Bottom line: Singleton is the guide's #3 pattern overall and the single strongest "must write it live, thread-safe, from memory" signal anywhere in the dataset — specifically for Malaysia. Everywhere else it's a near-certain supporting pattern, not usually the headline of a round.

निचोड़: Singleton गाइड का कुल मिलाकर #3 पैटर्न है और पूरे डेटासेट में कहीं भी सबसे मज़बूत अकेला "लाइव, थ्रेड-सेफ़, याद से लिखना ही होगा" संकेत है — खास तौर पर Malaysia के लिए। बाक़ी हर जगह यह एक लगभग-निश्चित सहायक पैटर्न है, आमतौर पर किसी राउंड की हेडलाइन नहीं।

Part 3 — Recognition, Decision Tree & When NOT to Use

Requirement phrases that signal Singleton: - "There should only ever be one instance of the logger/config/connection pool/cache manager in the whole application." - "Multiple parts of the system need to share the exact same [state/connection/counter] without passing it around explicitly." - "Avoid creating a new [expensive resource] every time it's needed." - "Provide a single, well-known access point to [X] from anywhere in the codebase."

Singleton का संकेत देने वाले शब्द/वाक्यांश: - "पूरे ऐप्लिकेशन में logger/config/connection pool/cache manager का हमेशा सिर्फ एक ही इंस्टेंस होना चाहिए।" - "सिस्टम के कई हिस्सों को बिना खुलकर इधर-उधर पास किए, बिल्कुल एक जैसी [स्थिति/कनेक्शन/काउंटर] साझा करनी है।" - "हर बार ज़रूरत पड़ने पर एक नया [महंगा संसाधन] बनाने से बचें।" - "कोडबेस में कहीं से भी [X] तक एक अकेला, जाना-पहचाना एक्सेस बिंदु प्रदान करें।"

Code smells that signal an existing Singleton opportunity (or an existing broken attempt at one): - A class is instantiated with `new` in dozens of places, always configured identically, and every instance behaves identically. - Global mutable state implemented as loose global variables or static class properties scattered across files, with no single owner. - A resource (DB connection, thread pool, file handle) is being opened and closed repeatedly where one long-lived instance would do.

कोड स्मेल्स जो Singleton के मौक़े (या टूटे हुए प्रयास) का संकेत देते हैं: - एक क्लास को दर्जनों जगहों पर `new` से इंस्टैंशिएट किया जाता है, हमेशा एक जैसा कॉन्फिगर्ड, और हर इंस्टेंस एक जैसा व्यवहार करता है। - वैश्विक परिवर्तनशील स्थिति ढीले वैश्विक वेरिएबल्स या फ़ाइलों में बिखरी स्टैटिक क्लास प्रॉपर्टीज़ के तौर पर लागू है, बिना किसी एक मालिक के। - एक संसाधन (DB कनेक्शन, थ्रेड पूल, फ़ाइल हैंडल) बार-बार खोला और बंद किया जाता है जहाँ एक लंबे समय तक जीवित इंस्टेंस काम आएगा।

Decision tree:

```
Does the requirement genuinely need exactly ONE instance for the
entire process lifetime (not per-request, not per-tenant, not per-user)?
|
├─ NO → it's scoped state, not Singleton. Use a container binding
      |   scoped to request/session/tenant instead.
|
├─ YES → Will this class ever need to be unit-tested in isolation,
      |   with this dependency mocked?
      |   |
      |   └─ YES (almost always, in production code) → Prefer a DI
      |         container "singleton" binding (Laravel's
      |         ->singleton(), Spring's default bean scope) –
      |         same "one instance, cached" behavior, but the
      |         class itself stays a plain, mockable class with
      |         no private constructor or static access point.
      |
      └─ NO / this is a small self-contained utility or a
            learning/interview exercise → Classic GoF
            Singleton (private constructor + static
            getInstance()) is acceptable.
```

निर्णय वृक्ष — हिंदी सार: पहले पूछें: क्या ज़रूरत को सचमुच पूरे प्रोसेस जीवनकाल के लिए ठीक एक इंस्टेंस चाहिए (प्रति-रिक्वेस्ट, प्रति-टेनेंट, प्रति-यूज़र नहीं)? अगर नहीं, तो यह स्कोप्ड स्टेट है, Singleton नहीं — इसके बजाय request/session/tenant पर स्कोप्ड

एक कंटेनर बाइंडिंग इस्तेमाल करें। अगर हाँ, तो पूछें: क्या इस क्लास को कभी अलग-थलग यूनिट-टेस्ट करना होगा, इस डिपेंडेंसी को मॉक करते हुए? अगर हाँ (प्रोडक्शन कोड में लगभग हमेशा), तो एक DI कंटेनर "singleton" बाइंडिंग को प्राथमिकता दें — वही "एक इंस्टेंस, कैश" व्यवहार, लेकिन क्लास खुद एक सादी, मॉक करने-योग्य क्लास बनी रहती है, बिना किसी प्राइवेट कंस्ट्रक्टर या स्टैटिक एक्सेस बिंदु के। अगर नहीं — यह एक छोटी स्वयं-निहित यूटिलिटी या एक सीखने/इंटरव्यू अभ्यास है — तो क्लासिक GoF Singleton (प्राइवेट कंस्ट्रक्टर + स्टैटिक `getInstance()`) स्वीकार्य है।

Explicit anti-triggers — do NOT reach for Singleton when: - You need a different configured instance per environment, tenant, or request (that's Factory, Registry, or a container-scoped binding — "one per X" is not "one, period"). - The class needs to be unit-tested with this dependency swapped for a test double — classic Singleton's hard-coded `getInstance()` call sites make that materially harder; a DI-container singleton binding solves the same problem without this cost. - You're using it purely to avoid passing a parameter through a few function calls — that's usually a sign the design needs restructuring (introduce a parameter object or a service layer), not a global. - In a distributed system with multiple processes/pods/nodes — a Singleton only guarantees "one instance per process." If the requirement is really "exactly one across the whole fleet" (e.g., a distributed lock, a leader-election result), you need a distributed coordination mechanism (Redis lock, ZooKeeper, etcd), not this pattern — Singleton solves the in-process case only, and confusing the two is a common interview trap.

स्पष्ट एंटी-ट्रिगर्स — Singleton का इस्तेमाल न करें जब: - आपको प्रति-एनवायरनमेंट, टेनेंट, या रिक्वेस्ट एक अलग कॉन्फिगर्ड इंस्टेंस चाहिए (यह Factory, Registry, या एक कंटेनर-स्कोपेड बाइंडिंग है — "प्रति-X एक" "बस एक" नहीं है)। - क्लास को इस डिपेंडेंसी को एक टेस्ट डबल से बदलकर यूनिट-टेस्ट करना है — क्लासिक Singleton के हार्ड-कोडेड `getInstance()` कॉल-साइट्स इसे काफी मुश्किल बनाते हैं; एक DI-कंटेनर singleton बाइंडिंग इस लागत के बिना वही समस्या हल करती है। - आप इसे सिर्फ कुछ फ़ंक्शन कॉल्स के ज़रिए एक पैरामीटर पास करने से बचने के लिए इस्तेमाल कर रहे हैं — यह आमतौर पर एक संकेत है कि डिज़ाइन को फिर से संरचित करने की ज़रूरत है, ग्लोबल की नहीं। - कई प्रोसेसेज़/पॉइंस/नोड्स वाले एक डिस्ट्रिब्यूटेड सिस्टम में — एक Singleton सिर्फ "प्रति-प्रोसेस एक इंस्टेंस" की गारंटी देता है। अगर ज़रूरत वाकई "पूरे प्लैट में ठीक एक" है, तो आपको एक डिस्ट्रिब्यूटेड समन्वय तंत्र चाहिए, यह पैटर्न नहीं।

Part 4 — Cheat Sheet & Multi-Length Pitch

One-page cheat sheet:

Aspect	Summary
Problem solved	Uncontrolled creation of a resource that should logically exist exactly once (config, logger, connection pool).
Mechanism	Private constructor, private static instance, public static accessor, guarded clone/deserialize.
Cost	Global mutable state, hidden dependencies, hard to unit-test, breaks under multi-process/distributed deployment, can hide poor design.
Benefit	Guaranteed single instance, lazy initialization, controlled global access without true global variables.

Aspect	Summary
Modern alternative	DI container singleton-scope binding — same behavior, testable, no <code>getInstance()</code> call sites baked into consumers.
PHP-specific gotcha	PHP-FPM's shared-nothing-per-request model means a naive Singleton is automatically "safe" within one request but resets every request — very different from a long-running Node.js/Swoole/RoadRunner process, where the same instance persists across many requests and concurrency actually matters.

हिंदी अनुवाद / Hindi Translation:

पहलू	सार
हल की गई समस्या	एक ऐसे संसाधन का अनियंत्रित निर्माण जो तार्किक रूप से ठीक एक बार अस्तित्व में होना चाहिए (config, logger, connection pool)।
तंत्र	प्राइवेट कंस्ट्रक्टर, प्राइवेट स्टैटिक इंस्टेंस, पब्लिक स्टैटिक एक्सेसर, गार्डेड clone/deserialize।
लागत	वैश्विक परिवर्तनशील स्थिति, छिपी हुई डिपेंडेंसीज़, यूनिट-टेस्ट करना मुश्किल, मल्टी-प्रोसेस/डिस्ट्रिब्यूटेड डिप्लॉयमेंट में टूटता है, खराब डिज़ाइन छिपा सकता है।
फ़ायदा	गारंटीड सिंगल इंस्टेंस, आलसी (lazy) इनिशियलाइज़ेशन, असली ग्लोबल वेरिएबल्स के बिना नियंत्रित वैश्विक पहुँच।
आधुनिक विकल्प	DI कंटेनर singleton-scope बाइंडिंग — वही व्यवहार, टेस्ट करने-योग्य, कंज्यूमर्स में <code>getInstance()</code> कॉल-साइट्स नहीं।
PHP-विशिष्ट गड़बड़ी	PHP-FPM का शेयर्ड-नर्थिंग-प्रति-रिक्वेस्ट मॉडल मतलब एक सीधा-सादा Singleton एक रिक्वेस्ट के अंदर अपने आप "सुरक्षित" है लेकिन हर रिक्वेस्ट पर रीसेट होता है — एक लंबे समय तक चलने वाली Node.js/Swoole/RoadRunner प्रोसेस से बहुत अलग, जहाँ वही इंस्टेंस कई रिक्वेस्ट्स में बना रहता है और concurrency असल में मायने रखती है।

30 seconds: "Singleton makes sure a class has exactly one instance for the whole app and gives you one place to reach it from — a private constructor plus a static `getInstance()` that builds the object once and hands back the same one every time after."

30 सेकंड: "Singleton सुनिश्चित करता है कि एक क्लास के पास पूरे ऐप के लिए ठीक एक इंस्टेंस हो और आपको उस तक पहुँचने की एक जगह देता है — एक प्राइवेट कंस्ट्रक्टर, साथ ही एक स्टैटिक `getInstance()` जो ऑब्जेक्ट एक बार बनाता है और हर बाद की बार वही वापस देता है।"

1 minute: "Singleton makes sure a class has exactly one instance for the whole app's lifetime, with one global access point. You do it with a private constructor so nobody can `new` it directly, a private static property holding the one instance, and a public static `getInstance()` that lazily creates it on first call and returns the cached one after that. You also block cloning and deserialization so those can't sneak a second instance into existence. It's the natural fit for things like a logger, a config manager, or a connection pool — anything that's genuinely supposed to exist exactly once. The catch is it introduces global state and makes the class hard to mock in tests, which is why most modern codebases get the same 'one instance, shared everywhere' behavior from a DI container's singleton-scope binding instead of the textbook pattern."

1 मिनट: "Singleton सुनिश्चित करता है कि एक क्लास के पास पूरे ऐप के जीवनकाल के लिए ठीक एक इंस्टेंस हो, एक वैश्विक एक्सेस बिंदु के साथ। आप यह एक प्राइवेट कंस्ट्रक्टर से करते हैं ताकि कोई इसे सीधे `new` न कर सके, एक इंस्टेंस रखने वाली एक प्राइवेट स्टैटिक प्रॉपर्टी, और एक पब्लिक स्टैटिक `getInstance()` जो पहली कॉल पर आलसी रूप से इसे बनाती है और बाद में कैश वाली लौटाती है। आप cloning और deserialization को भी ब्लॉक करते हैं ताकि वे चुपके से दूसरा इंस्टेंस न बना सकें। यह एक logger, एक config manager, या एक connection pool जैसी चीजों के लिए स्वाभाविक फिट है — कोई भी चीज़ जो सचमुच ठीक एक बार अस्तित्व में होनी चाहिए। पेच यह है कि यह वैश्विक स्थिति पेश करता है और क्लास को टेस्ट्स में मॉक करना मुश्किल बनाता है, इसलिए ज़्यादातर आधुनिक कोडबेस टेक्स्टबुक पैटर्न के बजाय DI कंटेनर की singleton-scope बाइंडिंग से वही 'एक इंस्टेंस, हर जगह साझा' व्यवहार पाते हैं।"

3 minutes: adds — the thread-safety story (why naive lazy init races under concurrency, what double-checked locking does and why Java needs `volatile` for it to actually be correct post-JDK5, and why PHP's per-request model changes the calculus versus a long-running Swoole/Node process); the testability cost in concrete terms (a `getInstance()` call baked into a consumer can't be swapped for a mock without a static-state reset hack); the distributed-systems caveat (Singleton is process-scoped, not fleet-scoped — don't confuse it with a distributed lock); and the Registry/Multiton distinction for when "one" isn't really the requirement.

3 मिनट: इसमें जोड़ें — थ्रेड-सेफ्टी कहानी (सीधा-सादा lazy init concurrency के तहत क्यों दौड़ता है, double-checked locking क्या करता है और Java को JDK5 के बाद इसके सही होने के लिए `volatile` की ज़रूरत क्यों है, और PHP का प्रति-रिक्वेस्ट मॉडल एक लंबे समय तक चलने वाली Swoole/Node प्रोसेस के मुकाबले गणना क्यों बदलता है); ठोस शब्दों में टेस्टेबिलिटी लागत (एक कंज्यूमर में पक्की `getInstance()` कॉल को बिना स्टैटिक-स्टेट-रीसेट हैक के मॉक से बदला नहीं जा सकता); डिस्ट्रिब्यूटेड-सिस्टम्स चेतावनी (Singleton प्रोसेस-स्कोप है, फ्लीट-स्कोप नहीं); और Registry/Multiton भेद जब "एक" असल में ज़रूरत नहीं है।

10 minutes: full pattern — everything above, plus walking the interviewer through a real example end-to-end (e.g., a `ConfigManager` in a Laravel-style app), naming the SOLID tension it creates (violates the spirit of Dependency Inversion by having consumers reach out to a concrete global rather than receiving an injected dependency), comparing it live against the DI-container alternative with a one-line trade-off for each, and closing with the specific interview follow-up an interviewer is likely to layer on: "how would you make this safe if we moved this to a multi-worker async server?"

10 मिनट: पूरा पैटर्न — ऊपर की हर बात, साथ ही इंटरव्यूअर को एक असली उदाहरण शुरू से अंत तक दिखाना (जैसे, एक Laravel-शैली ऐप में एक `ConfigManager`), जो SOLID तनाव यह पैदा करता है उसका नाम लेना, DI-कंटेनर विकल्प के मुकाबले इसकी लाइव तुलना करना, और इस ख़ास फ़ॉलो-अप के साथ समाप्त करना जो एक इंटरव्यूअर संभवतः जोड़ेगा: "अगर हम इसे एक मल्टी-वर्कर एसिंक सर्वर पर ले जाएँ तो आप इसे कैसे सुरक्षित बनाएँगे?"

Part 5 — Timed Mock Drill

Prompt (45-60 minutes, live-coding style — matches the Maybank/AirAsia format from Part 2): "Design a `ConfigManager` for a payments backend. Configuration is loaded once from environment/config files at startup and read frequently across the request lifecycle by many unrelated services (payment gateway client, logger, feature-flag checker). Loading config

is moderately expensive (file I/O + parsing) and must never produce two different in-memory copies with different values. Implement it, then be ready to discuss how your implementation behaves if we move this service from PHP-FPM to a long-running Swoole worker pool."

प्रॉम्प्ट (45-60 मिनट, लाइव-कोडिंग शैली): "एक पेमेंट्स बैकएंड के लिए एक `ConfigManager` डिज़ाइन करें। कॉन्फिगरेशन स्टार्टअप पर एनवायरनमेंट/कॉन्फिग फ़ाइलों से एक बार लोड होती है और रिक्वेस्ट जीवनचक्र में कई असंबंधित सेवाओं द्वारा बार-बार पढ़ी जाती है। कॉन्फिग लोड करना मध्यम रूप से महंगा है और कभी भी अलग-अलग मानों वाली दो इन-मेमोरी कॉपियाँ नहीं बनानी चाहिए। इसे इम्प्लीमेंट करें, फिर यह चर्चा करने के लिए तैयार रहें कि अगर हम इस सेवा को *PHP-FPM* से एक लंबे समय तक चलने वाले *Swoole* वर्कर पूल में ले जाएँ तो आपका इम्प्लीमेंटेशन कैसे व्यवहार करता है।"

Time-boxed sub-steps: 1. **0-5 min** — Restate the requirement, confirm "exactly one instance, process lifetime" is genuinely the ask (not per-request config), name the pattern out loud, state the trade-off you're accepting (global state / testability cost) before writing code. 2. **5-20 min** — Implement the naive version: private constructor, private static instance, public static `getInstance()`, lazy load on first call, block `__clone()` / `__wakeup()`. 3. **20-30 min** — Interviewer follow-up: "what happens under concurrent access?" — explain PHP-FPM's per-request isolation (no real race there), then explain what changes under Swoole/RoadRunner (shared process, concurrent coroutines/requests *can* race on first initialization), and implement or narrate a locking guard for that context. 4. **30-40 min** — Interviewer follow-up: "how would you unit-test a service that depends on this?" — walk through why `ConfigManager::getInstance()` baked into a consumer resists mocking, and what changing to constructor-injected + container-managed singleton scope would look like instead. 5. **40-55 min** — Interviewer follow-up: "we're scaling to multiple pods behind a load balancer — does your Singleton still guarantee one instance system-wide?" — correctly say no, it's process-scoped only, and name what *would* be needed for a true fleet-wide single-instance guarantee (external coordination, not this pattern). 6. **55-60 min** — Wrap: state the final design's trade-offs unprompted, don't wait to be asked.

समय-सीमित उप-चरण: 1. **0-5 मिनट** — ज़रूरत को दोहराएँ, पुष्टि करें कि "ठीक एक इंस्टेंस, प्रोसेस जीवनकाल" सचमुच माँग है (प्रति-रिक्वेस्ट कॉन्फिग नहीं), पैटर्न का नाम ज़ोर से बताएँ, कोड लिखने से पहले वह ट्रेड-ऑफ़ बताएँ जो आप स्वीकार कर रहे हैं (वैश्विक स्थिति / टेस्टेबिलिटी लागत)। 2. **5-20 मिनट** — सीधा-सादा वर्शन इम्प्लीमेंट करें: प्राइवेट कंस्ट्रक्टर, प्राइवेट स्टैटिक इंस्टेंस, पब्लिक स्टैटिक `getInstance()`, पहली कॉल पर आलसी लोड, `__clone()` / `__wakeup()` ब्लॉक करें। 3. **20-30 मिनट** — इंटरव्यूअर का फ़ॉलो-अप: "समवर्ती एक्सेस के तहत क्या होता है?" — PHP-FPM का प्रति-रिक्वेस्ट अलगाव समझाएँ (वहाँ कोई असली रेस नहीं), फिर बताएँ कि Swoole/RoadRunner के तहत क्या बदलता है, और उस संदर्भ के लिए एक लॉकिंग गार्ड इम्प्लीमेंट या बताएँ। 4. **30-40 मिनट** — इंटरव्यूअर का फ़ॉलो-अप: "आप इस पर निर्भर एक सेवा को कैसे यूनिट-टेस्ट करेंगे?" — बताएँ कि एक कंज़्यूमर में पक्की `ConfigManager::getInstance()` मॉकिंग का विरोध क्यों करती है, और इसके बजाय कंस्ट्रक्टर-इंजेक्टेड + कंटेनर-प्रबंधित singleton scope में बदलना कैसा दिखेगा। 5. **40-55 मिनट** — इंटरव्यूअर का फ़ॉलो-अप: "हम एक लोड बैलेंसर के पीछे कई पॉइंस तक स्केल कर रहे हैं — क्या आपका Singleton अब भी सिस्टम-व्यापी एक इंस्टेंस की गारंटी देता है?" — सही ढंग से कहें नहीं, यह सिर्फ़ प्रोसेस-स्कोप है, और बताएँ कि एक असली फ़्लीट-व्यापी सिंगल-इंस्टेंस गारंटी के लिए क्या चाहिए होगा। 6. **55-60 मिनट** — समाप्ति: बिना पूछे अंतिम डिज़ाइन के ट्रेड-ऑफ़्स बताएँ।

Self-grading rubric (score yourself honestly): - [] Named the pattern and its trade-off before writing code, not after. - [] Implemented lazy initialization correctly (not eager, unless you explicitly justified why eager is safer here). - [] Blocked `__clone()` and `__wakeup()` /

`__unserialize()` , not just the constructor. - [] Correctly distinguished PHP-FPM's per-request safety from Swoole/RoadRunner's shared-process risk, without conflating them. - [] Correctly explained the testability cost and named the DI-container alternative unprompted. - [] Correctly said "process-scoped, not fleet-scoped" when the multi-pod follow-up came — did not claim the Singleton solves distributed uniqueness.

स्वयं-ग्रेडिंग रूब्रिक (खुद को ईमानदारी से स्कोर करें): - [] कोड लिखने से पहले, बाद में नहीं, पैटर्न और उसका ट्रेड-ऑफ़ बताया। - [] आलसी इनिशियलाइज़ेशन सही ढंग से इम्प्लीमेंट किया (eager नहीं, जब तक खुलकर सही न ठहराया हो)। - [] सिर्फ़ कंस्ट्रक्टर नहीं, `__clone()` और `__wakeup()` / `__unserialize()` भी ब्लॉक किए। - [] PHP-FPM की प्रति-रिक्वेस्ट सुरक्षा को Swoole/RoadRunner के शेयर्ड-प्रोसेस जोखिम से सही ढंग से अलग किया, गड़मड़ किए बिना। - [] टेस्टेबिलिटी लागत सही ढंग से समझाई और बिना पूछे DI-कंटेनर विकल्प का नाम लिया। - [] मल्टी-पॉड फ़ॉलो-अप आने पर सही ढंग से कहा "प्रोसेस-स्कोपेड, प्लूट-स्कोपेड नहीं" — यह दावा नहीं किया कि Singleton डिस्ट्रिब्यूटेड यूनिकनेस हल करता है।

Part 6 — Pattern Recognition Drill

For each scenario: name the pattern, justify it in one sentence, then explicitly say why the two next-most-plausible patterns don't fit as well.

हर परिदृश्य के लिए: पैटर्न का नाम बताएँ, एक वाक्य में सही ठहराएँ, फिर खुलकर बताएँ कि अगले दो सबसे संभावित पैटर्न्स उतने अच्छे फ़िट क्यों नहीं बैठते।

1. **"Every microservice needs to read the same feature-flag values, and we want exactly one in-memory feature-flag cache per running process, refreshed every 60 seconds."** → Singleton (one cache instance per process, refreshed in place) — not Registry (there's only one flag-cache, not many named ones) and not a plain static class (the cache needs to hold mutable, refreshable state and be swappable in tests, which a static class can't do cleanly).

हिंदी: हर माइक्रोसर्विस को एक जैसी feature-flag वैल्यूज़ पढ़नी हैं, और हमें हर चल रही प्रोसेस के लिए ठीक एक इन-मेमोरी feature-flag कैश चाहिए, हर 60 सेकंड में रीफ़्रेश। → Singleton (प्रति-प्रोसेस एक कैश इंस्टेंस, वहीं रीफ़्रेश) — Registry नहीं (सिर्फ़ एक ही flag-cache है, कई नामित नहीं) और सादा स्टैटिक क्लास नहीं (कैश को परिवर्तनशील, रीफ़्रेश-योग्य स्थिति रखनी है और टेस्ट्स में बदली जा सकनी चाहिए)।

1. **"We need a factory that returns a `PaymentGateway` object, and which concrete gateway (Stripe, Razorpay, PayU) depends on the merchant's configured region."** → Factory Method, not Singleton — the requirement is "pick the right concrete type," not "ensure only one instance exists"; nothing here says a gateway object should be process-unique.

हिंदी: हमें एक factory चाहिए जो एक `PaymentGateway` ऑब्जेक्ट लौटाए, और कौन-सा कॉन्क्रीट गेटवे मर्चेन्ट के कॉन्फ़िगर्ड क्षेत्र पर निर्भर करता है। → Factory Method, Singleton नहीं — ज़रूरत है "सही कॉन्क्रीट टाइप चुनना," "सिर्फ़ एक इंस्टेंस सुनिश्चित करना" नहीं; यहाँ कुछ भी नहीं कहता कि एक गेटवे ऑब्जेक्ट प्रोसेस-अद्वितीय होना चाहिए।

1. **"Our test suite keeps failing intermittently because two tests running in the same process both mutate a shared `ConfigManager::getInstance()` and interfere**

with each other's expected values." → This is the *textbook cost* of Singleton showing up as a bug report, not a scenario to apply the pattern to — the fix is migrating `ConfigManager` to a DI-container-managed instance that tests can override or reset per-test, not doubling down on the static accessor.

हिंदी: हमारा टेस्ट सूट रुक-रुक कर विफल होता रहता है क्योंकि एक ही प्रोसेस में चल रहे दो टेस्ट्स दोनों एक साझा `ConfigManager::getInstance()` को बदलते हैं। → यह Singleton की टेक्स्टबुक लागत है जो एक बग रिपोर्ट के तौर पर सामने आ रही है, कोई परिदृश्य नहीं जहाँ पैटर्न लगाना है — फ़िक्स है `ConfigManager` को एक DI-कंटेनर-प्रबंधित इंस्टेंस में माइग्रेट करना, स्टैटिक एक्सेसर पर दोगुना दांव लगाना नहीं।

1. **"We need a connection pool that hands out a fixed number of reusable database connections, tracking which are checked out."** → Singleton *for the pool object itself* (there should be exactly one pool managing the connections) combined with Object Pool for the connection-reuse mechanics inside it — not Prototype (connections aren't cloned copies of each other, they're interchangeable pool members) and not Factory alone (a Factory would hand out a *new* connection every time, which defeats the pooling requirement).

हिंदी: हमें एक connection pool चाहिए जो पुनः प्रयोज्य डेटाबेस कनेक्शन्स की एक निश्चित संख्या बाँटे। → पूल ऑब्जेक्ट के लिए खुद Singleton (कनेक्शन्स को प्रबंधित करने वाला ठीक एक पूल होना चाहिए), अंदर कनेक्शन-पुनः इस्तेमाल तंत्र के लिए Object Pool के साथ मिलाकर — Prototype नहीं, अकेली Factory भी नहीं (एक Factory हर बार एक नया कनेक्शन बाँटेगी, जो पूलिंग ज़रूरत को हरा देती है)।

1. **"We need one logger instance per request, tagged with that request's correlation ID, not shared across requests."** → Not Singleton — "one per request" is scoped state, not "one, period"; this is a container binding scoped to request lifetime (Laravel's `scoped()` binding, for example), which is exactly the anti-trigger from Part 3.

हिंदी: हमें प्रति-रिक्वेस्ट एक logger इंस्टेंस चाहिए, उस रिक्वेस्ट की correlation ID से टैग किया गया, रिक्वेस्ट्स में साझा नहीं। → Singleton नहीं — "प्रति-रिक्वेस्ट एक" स्कोपेड स्थिति है, "बस एक" नहीं; यह रिक्वेस्ट जीवनकाल पर स्कोपेड एक कंटेनर बाइंडिंग है, जो ठीक Part 3 का एंटी-ट्रिगर है।

DEEP DIVE

Path map: `Fundamentals` → `Problem` → `Internals` → `Design` → `Implementation` → `Production` → `Trade-offs` → `Bugs` → `Interview Bank`.

पथ मानचित्र: बुनियाद → समस्या → आंतरिक कार्यप्रणाली → डिज़ाइन → इम्प्लीमेंटेशन → प्रोडक्शन → ट्रेड-ऑफ़्स → बग्स → इंटरव्यू बैंक।

Part 7 — Fundamentals

Definition: Singleton is a creational pattern that restricts instantiation of a class to a single object and provides a global point of access to that object.

परिभाषा: Singleton एक क्रिएशनल पैटर्न है जो एक क्लास के इंस्टैंशिएशन को एक अकेले ऑब्जेक्ट तक सीमित करता है और उस ऑब्जेक्ट तक पहुँच का एक वैश्विक बिंदु प्रदान करता है।

Beginner framing: you want exactly one `Logger`, so instead of trusting every developer to remember "don't create a second one," the class refuses to let anyone create it directly — you always ask the class itself for "the" instance, and it either builds one the first time or hands you back the one it already built.

शुरुआती स्तर की समझ: आपको ठीक एक `Logger` चाहिए, इसलिए हर डेवलपर पर यह भरोसा करने के बजाय कि वह याद रखेगा "दूसरा मत बनाना," क्लास किसी को भी इसे सीधे बनाने से मना कर देती है — आप हमेशा क्लास से ही "वह" इंस्टेंस माँगते हैं, और यह पहली बार एक बनाती है या जो पहले से बना चुकी है वह वापस देती है।

Senior/staff framing: Singleton is really solving two separable problems that GoF bundled into one pattern — **(1) controlled, lazy creation of an expensive or stateful resource**, and **(2) global accessibility without passing the reference through every layer of the call stack**. Modern practice has largely un-bundled these: DI containers solve (1) cleanly via singleton-scoped bindings, and (2) is now widely treated as a smell to *avoid* rather than a feature to lean on, because global reachability is exactly what makes a class's dependencies implicit and its tests fragile. Knowing this bundling — and why senior engineers reach for the container instead of the textbook pattern — is itself a strong interview signal.

सीनियर/स्टाफ़ स्तर की समझ: Singleton असल में दो अलग करने-योग्य समस्याएँ हल करता है जिन्हें GoF ने एक पैटर्न में बांध दिया — **(1) एक महँगे या स्थिति-वाले संसाधन का नियंत्रित, आलसी निर्माण**, और **(2) कॉल स्टैक की हर परत से रेफ़रेंस पास किए बिना वैश्विक पहुँच**। आधुनिक व्यवहार ने इन्हें काफ़ी हद तक अलग कर दिया है: DI कंटेनर्स singleton-scoped बाइंडिंग्स के ज़रिए (1) को साफ़ तौर पर हल करते हैं, और (2) को अब व्यापक रूप से एक ऐसी बुराई माना जाता है जिससे बचना चाहिए, न कि एक फ़ीचर जिस पर टिका जाए, क्योंकि वैश्विक पहुँच ही वह चीज़ है जो एक क्लास की डिपेंडेंसीज़ को छिपा हुआ और उसके टेस्ट्स को नाज़ुक बनाती है। यह बंधन जानना — और सीनियर इंजीनियर टेक्स्टबुक पैटर्न के बजाय कंटेनर की ओर क्यों पहुँचते हैं — खुद एक मज़बूत इंटरव्यू संकेत है।

Part 8 — The Engineering Problem & Refactoring Trigger

What code looks like before this pattern: a `ConfigManager` (or `Logger`, or `DbConnection`) gets instantiated with `new` at multiple call sites across the codebase — a controller here, a background job there, a CLI command somewhere else. Each `new ConfigManager()` re-reads and re-parses the same config files from disk. Worse, if `ConfigManager` holds any runtime-mutable state (a feature-flag override set at runtime, for instance), each instance now has its own copy, and different parts of the app silently disagree about the current config.

यह पैटर्न लगाने से पहले कोड कैसा दिखता है: एक `ConfigManager` (या `Logger`, या `DbConnection`) कोडबेस में कई कॉल-साइट्स पर `new` से इंस्टैंशिएट होता है — यहाँ एक कंट्रोलर, वहाँ एक बैकग्राउंड जॉब, कहीं और एक CLI कमांड। हर `new ConfigManager()` डिस्क से वही कॉन्फिग फ़ाइलें फिर से पढ़ता और पार्स करता है। बदतर, अगर `ConfigManager` कोई रनटाइम-परिवर्तनशील स्थिति रखता है, तो हर इंस्टेंस की अब अपनी खुद की कॉपी है, और ऐप के अलग-अलग हिस्से मौजूदा कॉन्फिग को लेकर चुपचाप असहमत हैं।

Why it breaks down at scale: the cost compounds two ways — wasted work (re-parsing the same files repeatedly) and correctness risk (multiple sources of truth for state that must be consistent). The failure mode is rarely a crash; it's a silent inconsistency bug that surfaces as "why did this request see the old feature-flag value" days later.

यह बड़े पैमाने पर क्यों टूटता है: लागत दो तरीकों से बढ़ती है — बर्बाद काम (वही फ़ाइलें बार-बार पार्स करना) और सटीकता जोखिम (एक जैसी रहनी चाहिए वाली स्थिति के लिए कई सच्चाई के स्रोत)। विफलता ढंग शायद ही कभी क्रैश होता है; यह एक खामोश असंगति बग है जो दिनों बाद "इस रिक्वेस्ट को पुराना feature-flag मान क्यों दिखा" के तौर पर सामने आती है।

The code smell that should make an engineer reach for it: repeated `new SomeExpensiveOrStatefulThing()` calls, all configured identically, scattered across files that have no reason to know about each other's instantiation.

कोड स्मेल जो एक इंजीनियर को इसकी ओर ले जाना चाहिए: दोहराई गई `new SomeExpensiveOrStatefulThing()` कॉल्स, सभी एक जैसी कॉन्फिगर्ड, उन फ़ाइलों में बिखरी जिन्हें एक-दूसरे के इंस्टैंशिएशन के बारे में जानने की कोई ज़रूरत नहीं।

Production-mindset questions: - *What production problem actually forced engineers toward this pattern?* — usually either a measurable cost (re-opening DB connections or re-parsing config on every use) or a correctness bug (two components silently disagreeing about shared state) rather than a purely theoretical concern. - *How would a senior engineer discover the requirement before it became a crisis?* — a code review flagging repeated `new X()` for something that's conceptually "the app's one and only Y," or a profiler/APM trace showing redundant I/O from repeated construction. - *What metric would have shown it coming?* — elevated request latency or DB connection-pool exhaustion correlated with connection object churn; config-parsing time showing up disproportionately in a flame graph. - *What alternatives would a competent engineer consider and reject first?* — "just document that everyone should reuse one instance" (relies on discipline, doesn't scale with team size); a plain global variable (works but has none of Singleton's lazy-init or encapsulation benefits and is even harder to reason about); passing the instance explicitly through every constructor (the "correct" DI

answer, often rejected early only because of the refactor size, then revisited later once the codebase matures).

प्रोडक्शन-सोच वाले सवाल: - असल में किस प्रोडक्शन समस्या ने इंजीनियरों को इस पैटर्न की ओर मजबूर किया? — आमतौर पर या तो एक मापने-योग्य लागत (हर इस्तेमाल पर DB कनेक्शन फिर से खोलना या कॉन्फ़िग फिर से पार्स करना) या एक सटीकता बग (दो कंपोनेंट्स साझा स्थिति को लेकर चुपचाप असहमत)। - एक सीनियर इंजीनियर संकट बनने से पहले इस ज़रूरत को कैसे खोजेगा? — एक कोड रिव्यू जो किसी ऐसी चीज़ के लिए दोहराई गई `new X()` को चिह्नित करे जो संकल्पनात्मक रूप से "ऐप की एक और अकेली Y" है, या एक प्रोफ़ाइलर/APM ट्रेस जो दोहराए गए निर्माण से अनावश्यक I/O दिखाए। - कौन-सा मेट्रिक इसे आते हुए दिखाता? — कनेक्शन ऑब्जेक्ट चर्न से संबंधित बढ़ी हुई रिक्वेस्ट लेटेंसी या DB कनेक्शन-पूल थकावट; एक प्रलेम ग्राफ़ में असंगत रूप से दिखता कॉन्फ़िग-पार्सिंग समय। - एक सक्षम इंजीनियर पहले किन विकल्पों पर विचार करके अस्वीकार करेगा? — "बस दस्तावेज़ीकृत करो कि सभी को एक इंस्टेंस पुनः इस्तेमाल करना चाहिए" (अनुशासन पर निर्भर करता है); एक सादा ग्लोबल वेरिएबल (काम करता है लेकिन Singleton के आलसी-इनिट या एनकैप्सुलेशन फ़ायदों में से कोई नहीं); इंस्टेंस को हर कंस्ट्रक्टर के ज़रिए खुलकर पास करना (सही DI जवाब, अक्सर रीफ़ैक्टर के आकार की वजह से शुरू में अस्वीकार किया जाता है, कोडबेस परिपक्व होने पर बाद में दोबारा देखा जाता है)।

(Full before/after refactoring code sequence lives in Part 19, not duplicated here.)

(पूरा पहले/बाद रीफ़ैक्टरिंग कोड क्रम Part 19 में है, यहाँ दोहराया नहीं गया।)

Part 9 — Internal Working

Concept level (language-agnostic): the pattern relies on the language allowing you to (a) restrict who can call a constructor, and (b) attach state and behavior to the *class itself* rather than only to instances — i.e., static/class-level members. `getInstance()` performs a check-then-act sequence: "does the cached instance exist yet? If not, create and cache it; either way, return it." That check-then-act sequence is the entire source of every concurrency bug this pattern can produce — two threads/coroutines can both pass the "does it exist?" check before either finishes creating it, producing two instances.

संकल्पना स्तर (भाषा-निरपेक्ष): पैटर्न इस बात पर निर्भर करता है कि भाषा आपको (a) यह प्रतिबंधित करने दे कि कौन कंस्ट्रक्टर कॉल कर सकता है, और (b) स्थिति और व्यवहार को सिर्फ़ इंस्टेंसेज़ के बजाय खुद क्लास से जोड़ने दे — यानी स्टैटिक/क्लास-स्तर के सदस्य। `getInstance()` एक जाँच-फिर-कार्य क्रम करता है: "क्या कैशेड इंस्टेंस अभी मौजूद है? अगर नहीं, तो बनाओ और कैश करो; दोनों ही स्थिति में, इसे लौटाओ।" वह जाँच-फिर-कार्य क्रम ही इस पैटर्न से पैदा होने वाली हर concurrency बग का पूरा स्रोत है — दो थ्रेड्स/कोरूटीन्स दोनों "क्या यह मौजूद है?" जाँच को पास कर सकते हैं इससे पहले कि कोई इसे बनाना पूरा करे, दो इंस्टेंसेज़ पैदा करते हुए।

PHP-specific mechanics, only as deep as this pattern's core gotcha requires: - Under classic **PHP-FPM**, each incoming HTTP request gets a fresh PHP process/worker with its own memory space — nothing (including your Singleton's static property) survives between requests, and nothing is shared *during* a request across threads, because there's only one execution context handling that request. This means the naive, non-thread-safe `getInstance()` is completely safe under PHP-FPM — but also means your "singleton" is really "one instance per request," which is often *not* what a config manager conceptually wants (it just happens not to matter for correctness, since each request re-derives the same config). - Under a **long-running process model — Swoole, RoadRunner, or a persistent worker in Node.js/Java-style deployments** — the process (and therefore the static instance) persists across

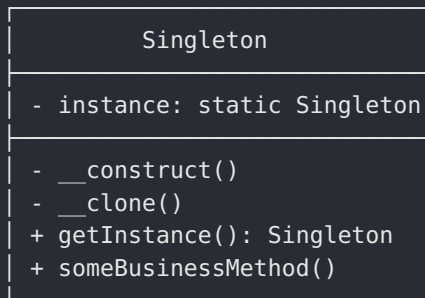
many requests, and multiple requests/coroutines can be in flight concurrently within that same process. Now the check-then-act race in `getInstance()` is real: two coroutines can both observe "not yet created" before either finishes constructing the instance, and you can end up with two live instances silently coexisting — exactly the bug the pattern was supposed to prevent. - **PHP has no native `volatile` keyword or memory-visibility primitive** the way Java does — this is a genuine language-level gap. On a long-running PHP server (Swoole), the safe approaches are: use Swoole's own coroutine-safe primitives (a `Swoole\Lock` or `Swoole\Coroutine\Channel`-based guard around initialization), or — more commonly in practice — sidestep the problem entirely by initializing the singleton once at worker-boot time (before any request-handling coroutines start), rather than lazily on first access. This is a real, current PHP-ecosystem gotcha, not a theoretical one, since Swoole/RoadRunner adoption for high-throughput PHP services has grown specifically because of this shared-process performance model.

PHP-विशिष्ट कार्यप्रणाली, सिर्फ़ इतनी गहराई जितनी इस पैटर्न की मुख्य गड़बड़ी को चाहिए: - क्लासिक **PHP-FPM** के तहत, हर आने वाली HTTP रिक्वेस्ट को अपनी खुद की मेमोरी स्पेस वाली एक ताज़ा PHP प्रोसेस/वर्कर मिलती है — कुछ भी (आपके Singleton की स्टैटिक प्रॉपर्टी सहित) रिक्वेस्ट्स के बीच नहीं बचता। इसका मतलब है कि सीधा-सादा, गैर-थ्रेड-सेफ़ `getInstance()` PHP-FPM के तहत पूरी तरह सुरक्षित है — लेकिन इसका यह भी मतलब है कि आपका "singleton" असल में "प्रति-रिक्वेस्ट एक इंस्टेंस" है, जो अक्सर वह नहीं है जो एक config manager संकल्पनात्मक रूप से चाहता है। - एक **लंबे समय तक चलने वाले प्रोसेस मॉडल — Swoole, RoadRunner, या Node.js/Java-शैली डिप्लॉयमेंट्स में एक स्थायी वर्कर** — के तहत प्रोसेस (और इसलिए स्टैटिक इंस्टेंस) कई रिक्वेस्ट्स में बना रहता है, और कई रिक्वेस्ट्स/कोरूटीन्स उसी प्रोसेस के अंदर समवर्ती रूप से चल सकते हैं। अब `getInstance()` में जाँच-फिर-कार्य रेस असली है: दो कोरूटीन्स दोनों "अभी नहीं बना" देख सकते हैं इससे पहले कि कोई इंस्टेंस बनाना पूरा करे, और आप चुपचाप सह-अस्तित्व में दो जीवित इंस्टेंसेज़ पा सकते हैं। - **PHP के पास Java जैसा कोई मूल `volatile` कीवर्ड या मेमोरी-विज़िबिलिटी प्रिमिटिव नहीं है** — यह एक असली भाषा-स्तर का अंतराल है। एक लंबे समय तक चलने वाले PHP सर्वर (Swoole) पर, सुरक्षित तरीक़े हैं: Swoole के अपने कोरूटीन-सेफ़ प्रिमिटिव्स इस्तेमाल करना, या — व्यवहार में ज़्यादा आम — इस समस्या को पूरी तरह टालना, वर्कर-बूट समय पर एक बार सिंगलटन को इनिशियलाइज़ करके पहली एक्सेस पर आलसी रूप से नहीं। यह एक असली, मौजूदा PHP-इकोसिस्टम गड़बड़ी है, सैद्धांतिक नहीं।

Part 10 — Components, UML & Language Mapping

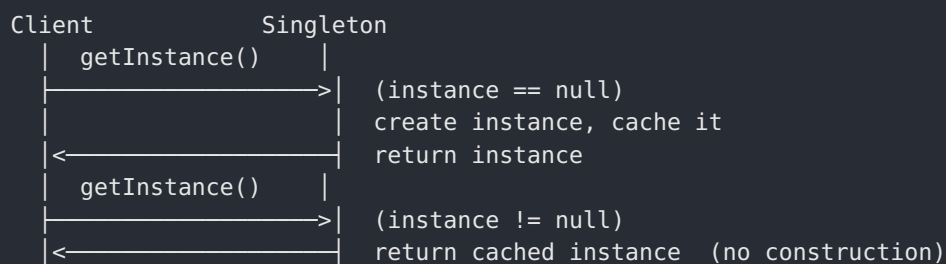
Roles: - **Singleton (the class itself):** owns a private static reference to its one instance, a private/protected constructor, and a public static accessor. - **Client:** never calls `new`; always goes through the static accessor.

भूमिकाएँ: - **Singleton (क्लास खुद):** अपने एक इंस्टेंस के लिए एक प्राइवेट स्टैटिक रेफ़रेंस, एक प्राइवेट/प्रोटेक्टेड कंस्ट्रक्टर, और एक पब्लिक स्टैटिक एक्सेसर रखती है। - **Client:** कभी `new` कॉल नहीं करता; हमेशा स्टैटिक एक्सेसर से गुज़रता है।



Sequence (first call vs. subsequent call) — worth drawing because it's the one diagram that actually explains the lazy-init behavior an interviewer will ask about:

अनुक्रम (पहली कॉल बनाम बाद की कॉल) — बनाने लायक क्योंकि यह वह एक डायग्राम है जो lazy-init व्यवहार को असल में समझाता है:



Language mapping for the core mechanism:

Language	How "exactly one instance" is typically achieved
PHP 8.3	Private constructor + private static property + public static <code>getInstance()</code> ; block <code>__clone()</code> (throw) and <code>__wakeup()</code> / <code>__unserialize()</code> .
Java	Same shape, but thread-safety is a first-class concern — either <code>enum Singleton { INSTANCE }</code> (JVM-guaranteed single instantiation, the community-preferred modern approach), or double-checked locking with a <code>volatile</code> instance field.
Python	Rarely done via <code>__new__</code> override in production code — more commonly a module-level instance (Python modules are already singletons, imported once and cached by the interpreter) or a metaclass-based singleton for a class hierarchy.
Go	<code>sync.Once</code> — <code>Do()</code> guarantees the initialization function runs exactly once even under concurrent callers, which is a cleaner primitive than manual double-checked locking.
TypeScript/Node.js	Node's module cache already behaves like a singleton for anything exported from a module (imported once, same object reference everywhere) — an explicit class-based Singleton is less common than in PHP/Java for this reason, similar to Python.

हिंदी सार (भाषा-मानचित्रण): PHP 8.3 — प्राइवेट कंस्ट्रक्टर + प्राइवेट स्टैटिक प्रॉपर्टी + पब्लिक स्टैटिक `getInstance()` ; `__clone()` (थ्रो) और `__wakeup()` / `__unserialize()` ब्लॉक करें। Java — वही आकार, लेकिन थ्रेड-सेफ्टी एक प्रमुख

चिंता है — या तो `enum Singleton { INSTANCE }` (JVM-गारंटीड सिंगल इंस्टैंशिएशन), या `volatile` इंस्टेंस फ़ील्ड के साथ double-checked locking। Python — प्रोडक्शन कोड में `__new__` ओवरराइड के ज़रिए शायद ही कभी किया जाता है — ज़्यादा आमतौर पर एक मॉड्यूल-स्तर इंस्टेंस। Go — `sync.Once` — `Do()` गारंटी देता है कि इनिशियलाइज़ेशन फ़ंक्शन ठीक एक बार चलता है। TypeScript/Node.js — Node का मॉड्यूल कैश पहले से ही एक मॉड्यूल से एक्सपोर्ट की गई किसी भी चीज़ के लिए एक सिंगलटन की तरह व्यवहार करता है।

Part 11 — Implementation Overview (PHP/Laravel/Node)

The companion `Singleton.php` file walks through: a naive (not-concurrency-safe) version; a version with clone/wakeup guards; a Swoole-aware version using boot-time initialization instead of lazy check-then-act; and a `ConfigManager` real-world example.

साथी `Singleton.php` फ़ाइल इनसे गुज़रती है: एक सीधा-सादा (concurrency-सेफ़ नहीं) वर्शन; clone/wakeup गार्ड्स वाला एक वर्शन; lazy check-then-act के बजाय boot-time इनिशियलाइज़ेशन इस्तेमाल करने वाला एक Swoole-जागरूक वर्शन; और एक `ConfigManager` असली-दुनिया उदाहरण।

Where this pattern genuinely does — and doesn't — show up in real framework internals, verified against current source rather than assumed:

- **Laravel's service container** `->singleton()` **binding** is the idiomatic modern replacement most PHP teams actually reach for instead of the textbook pattern. Per Laravel's own documentation (verified against the current Service Container docs): `singleton()` registers a binding that is "resolved only one time" — the first time the container resolves it, it builds the instance and caches it internally; every subsequent resolution of that binding returns the same cached object. Contrast with `bind()`, which constructs a fresh instance on every resolution. Critically, this achieves the "one instance" guarantee **without** a private constructor or a `getInstance()` call baked into consumers — the class stays a plain, constructor-injectable, mockable PHP class, and the container is the only thing that knows it's being treated as a singleton. This is the textbook illustration of Part 7's "un-bundling" point: Laravel gives you problem (1) — controlled single instantiation — without problem (2)'s global-accessibility cost.
- **Spring Boot's default bean scope is `singleton`** (one instance per Spring `ApplicationContext`, not per JVM) — the same idea, same un-bundling, for readers whose interviews may run in Java/Spring shops (Oracle India and Mastercard India per Part 2's data both evaluate Spring-adjacent backend roles).
- **What Singleton is *not*, in framework terms:** it's easy to conflate "the framework's container treats this as a singleton" with "this class implements the GoF Singleton pattern" — they produce the same runtime behavior (one shared instance) through opposite mechanisms (container-managed lifetime vs. self-enforced construction control). An interviewer asking "does Laravel use Singleton internally?" is testing whether you know this distinction, not just whether you can define the pattern.

यह पैटर्न असली फ़्रेमवर्क आंतरिक कार्यप्रणाली में सचमुच कहाँ दिखता है — और कहाँ नहीं:

- **Laravel के सर्विस कंटेनर की `->singleton()` बाइंडिंग** वह मुहावरेदार आधुनिक विकल्प है जिसकी ओर ज़्यादातर PHP टीमों टेक्स्टबुक पैटर्न के बजाय असल में पहुँचती हैं। Laravel के अपने दस्तावेज़ीकरण के अनुसार (मौजूदा Service Container docs के खिलाफ़ सत्यापित): `singleton()` एक ऐसी बाइंडिंग रजिस्टर करता है जो "सिर्फ़ एक बार रिज़ॉल्व" होती है — कंटेनर पहली बार इसे रिज़ॉल्व करने पर

इंस्टेंस बनाता है और आंतरिक रूप से कैश करता है; उस बाइंडिंग का हर बाद का रिज़ॉल्यूशन वही कैश ऑब्जेक्ट लौटाता है। `bind()` से इसका उलट, जो हर रिज़ॉल्यूशन पर एक ताज़ा इंस्टेंस बनाता है। महत्वपूर्ण रूप से, यह "एक इंस्टेंस" गारंटी को **बिना** किसी प्राइवेट कंस्ट्रक्टर या कंज्यूमर्स में पक्की `getInstance()` कॉल के हासिल करता है — क्लास एक सादी, कंस्ट्रक्टर-इंजेक्टेबल, मॉक करने-योग्य PHP क्लास बनी रहती है। - **Spring Boot का डिफ़ॉल्ट बीन स्कोप `singleton` है** (प्रति `Spring ApplicationContext` एक इंस्टेंस, प्रति JVM नहीं) — वही विचार, वही अलगाव, उन पाठकों के लिए जिनके इंटरव्यूज़ Java/Spring दुकानों में चल सकते हैं। - **Singleton क्या नहीं है, फ्रेमवर्क शब्दों में:** "फ्रेमवर्क का कंटेनर इसे एक singleton की तरह मानता है" को "यह क्लास GoF Singleton पैटर्न इम्प्लीमेंट करती है" के साथ गड़मड़ करना आसान है — वे विपरीत तंत्रों (कंटेनर-प्रबंधित जीवनकाल बनाम स्वयं-लागू निर्माण नियंत्रण) के ज़रिए एक जैसा रनटाइम व्यवहार (एक साझा इंस्टेंस) पैदा करते हैं।

Part 12 — Where This Shows Up in Production

Scenario 1 — Payments platform config manager (Razorpay/Postman-style, per Part 2's India Tier-2 data): a `ConfigManager` loads API keys, feature flags, and rate-limit thresholds once at boot and serves them to dozens of unrelated services. Implemented as a Laravel container singleton binding rather than a classic static-accessor Singleton specifically so the payment-gateway integration tests could inject a fake config without touching global state.

परिदृश्य 1 — पेमेंट्स प्लेटफ़ॉर्म कॉन्फ़िग मैनेजर (Razorpay/Postman-शैली): एक `ConfigManager` API keys, feature flags, और rate-limit थ्रेशोल्ड्स को बूट पर एक बार लोड करता है और दर्जनों असंबंधित सेवाओं को परोसता है। एक क्लासिक स्टैटिक-एक्सेसर Singleton के बजाय एक Laravel कंटेनर singleton बाइंडिंग के तौर पर इम्प्लीमेंट किया गया, खास तौर पर ताकि पेमेंट-गेटवे इंटीग्रेशन टेस्ट्स वैश्विक स्थिति को छुए बिना एक नक़ली कॉन्फ़िग इंजेक्ट कर सकें।

Scenario 2 — Ride-hailing dispatch service connection pool (Uber India/Grab-style, per Part 2's Tier-1 data): a single `DbConnectionPool` instance manages a fixed set of reusable database connections shared across the dispatch service's request handlers running under a long-running worker process. This is the scenario where the Swoole/RoadRunner concurrency caveat from Part 9 is not academic — the pool's initialization genuinely needs a concurrency guard because the worker process is shared across many in-flight coroutines.

परिदृश्य 2 — राइड-हेलिंग डिस्पैच सेवा कनेक्शन पूल (Uber India/Grab-शैली): एक अकेला `DbConnectionPool` इंस्टेंस पुनः प्रयोज्य डेटाबेस कनेक्शन्स के एक निश्चित सेट को प्रबंधित करता है, एक लंबे समय तक चलने वाली वर्कर प्रोसेस के तहत चलने वाले डिस्पैच सेवा के रिक्वेस्ट हैंडलर्स में साझा। यह वह परिदृश्य है जहाँ Part 9 की Swoole/RoadRunner concurrency चेतावनी अकादमिक नहीं है — पूल के इनिशियलाइज़ेशन को सचमुच एक concurrency गार्ड चाहिए।

Scenario 3 — Digital banking audit logger (Maybank-style, per Part 2's Malaysia data): a single `AuditLogger` instance ensures every audit-trail write goes through one code path with consistent formatting and a single open file handle/log-shipping connection, avoiding interleaved or duplicated log entries that multiple independent logger instances could produce.

परिदृश्य 3 — डिजिटल बैंकिंग ऑडिट लॉगर (Maybank-शैली): एक अकेला `AuditLogger` इंस्टेंस सुनिश्चित करता है कि हर audit-trail राइट एक ही कोड पथ से गुज़रे, संगत फ़ॉर्मेटिंग और एक अकेली खुली फ़ाइल हैंडल/लॉग-शिपिंग कनेक्शन के साथ, कई स्वतंत्र logger इंस्टेंसेज़ पैदा कर सकते इंटरलीव्ड या दोहराई गई log एंट्रीज़ से बचते हुए।

Microservices-usage table:

Component	Typically Singleton-shaped?	Why
Config/feature-flag manager	Yes	Genuinely one source of truth per process.
Structured logger	Yes	One consistent output stream/format per process.
DB connection pool	Yes (the pool object; not each connection)	Pooling requires one coordinator.
Cache client wrapper (Redis/Memcached connection)	Usually	Avoids reconnect overhead per use.
Per-request correlation-ID context	No	Scoped to request, not process — common anti-trigger from Part 3.
Domain entities (User, Order, Payment)	No	Each is naturally many-instantiated; forcing Singleton here is a design smell.

हिंदी सार: Config/feature-flag मैनेजर — हाँ (सचमुच प्रति-प्रोसेस एक सच्चाई का स्रोत)। संरचित logger — हाँ (प्रति-प्रोसेस एक संगत आउटपुट)। DB कनेक्शन पूल — हाँ, पूल ऑब्जेक्ट के लिए, हर कनेक्शन के लिए नहीं। Cache क्लाइंट रैपर — आमतौर पर हाँ। प्रति-रिक्वेस्ट correlation-ID संदर्भ — नहीं (रिक्वेस्ट पर स्कोप, प्रोसेस पर नहीं)। डोमेन एंटीटीज़ (User, Order, Payment) — नहीं (हर एक स्वाभाविक रूप से कई-इंस्टेंस वाली है; यहाँ Singleton थोपना एक डिज़ाइन खामी है)।

Architecture Decision Record — adopting a container-managed singleton for ConfigManager :

आर्किटेक्चर डिज़ीज़न रिकॉर्ड — ConfigManager के लिए एक कंटेनर-प्रबंधित singleton अपनाना:

- **Context:** ConfigManager was being instantiated with `new` in 14 call sites across the payments service, each re-parsing the same YAML config files, and two integration tests had started failing intermittently due to config drift between instances created at slightly different times during a deploy.
- **संदर्भ:** ConfigManager को पेमेंट्स सेवा में 14 कॉल-साइट्स पर `new` से इंस्टैंशिएट किया जा रहा था, हर एक वही YAML कॉन्फ़िग फ़ाइलें फिर से पार्स करता था, और दो इंटीग्रेशन टेस्ट्स रुक-रुक कर विफल होने लगे थे।
- **Decision:** Register ConfigManager as a Laravel container singleton binding rather than implementing the GoF static-accessor pattern.
- **फ़ैसला:** GoF स्टैटिक-एक्सेसर पैटर्न इम्प्लीमेंट करने के बजाय ConfigManager को एक Laravel कंटेनर singleton बाइंडिंग के तौर पर रजिस्टर करें।
- **Consequences:** Config is now parsed exactly once per process; all 14 call sites were refactored to constructor-inject ConfigManager instead of instantiating it; integration tests can now bind a fake ConfigManager in the test container without touching production code.
- **नतीजे:** Config अब प्रति-प्रोसेस ठीक एक बार पार्स होती है; सभी 14 कॉल-साइट्स को constructor-inject करने के लिए रीफैक्टर किया गया; इंटीग्रेशन टेस्ट्स अब प्रोडक्शन कोड छुए बिना एक नकली ConfigManager बाइंड कर सकते हैं।

- **Alternatives considered:** (a) classic GoF Singleton — rejected because it would keep the same testability cost; (b) a plain global variable — rejected as strictly worse; (c) passing `ConfigManager` explicitly through every constructor — rejected only due to refactor size.
- **विचार किए गए विकल्प:** (a) क्लासिक GoF Singleton — अस्वीकार क्योंकि यह वही टेस्टेबिलिटी लागत बनाए रखता; (b) एक सादा ग्लोबल वेरिएबल — सख्ती से बदतर के तौर पर अस्वीकार; (c) `ConfigManager` को खुलकर हर कंस्ट्रक्टर से पास करना — सिर्फ़ रीफ़ैक्टर के आकार की वजह से अस्वीकार।
- **Trade-offs:** the team accepted that `ConfigManager` is still implicitly "special" rather than the container enforcing true single-instantiation the way a private constructor would.
- **ट्रेड-ऑफ़्स:** टीम ने स्वीकार किया कि `ConfigManager` अब भी छिपे तौर पर "खास" है, न कि कंटेनर वैसा सख्त सिंगल-इंस्टैंशिएशन लागू कर रहा जैसा एक प्राइवेट कंस्ट्रक्टर करता।

Part 13 — Field Notes (Simulated Production Experience)

Rehearsal scaffold, not a script — personalize with real project details before using as an actual interview answer, or present it plainly as illustrative rather than personal history.

रिहर्सल ढाँचा, कोई स्क्रिप्ट नहीं — असली इंटरव्यू जवाब के तौर पर इस्तेमाल करने से पहले असली प्रोजेक्ट विवरणों के साथ निजीकृत करें।

"On a payments team I worked with, we had a classic-style `Logger::getInstance()` singleton that had been in the codebase for years. It worked fine until we started running integration tests in parallel inside the same PHP-FPM worker pool during CI — turned out two tests could, under specific timing, both trigger `Logger`'s first-call initialization at nearly the same moment, and depending on scheduling we'd occasionally get a log file handle opened twice, with one handle silently going stale. The fix wasn't a locking mechanism — it was recognizing that a classic Singleton was solving a problem we no longer had, while creating a testability problem we did have. We migrated it to a container-managed singleton binding, injected everywhere it was needed, and the flaky test disappeared along with about 40 lines of defensive locking code that had never actually been the right fix in the first place."

"एक पेमेंट्स टीम में जिसके साथ मैंने काम किया, हमारे पास एक क्लासिक-शैली का `Logger::getInstance()` सिंगलटन था जो सालों से कोडबेस में था। यह ठीक चलता रहा जब तक हमने CI के दौरान उसी PHP-FPM वर्कर पूल के अंदर समानांतर इंटीग्रेशन टेस्ट्स चलाना शुरू नहीं किया — पता चला कि विशिष्ट समय के तहत, दो टेस्ट्स दोनों `Logger` का पहला-कॉल इनिशियलाइज़ेशन लगभग एक ही पल में ट्रिगर कर सकते थे, और शेड्यूलिंग के आधार पर कभी-कभी एक log फ़ाइल हैडल दो बार खुल जाता, एक हैडल चुपचाप बासी हो जाता। फ़िक्स एक लॉकिंग तंत्र नहीं था — यह पहचानना था कि एक क्लासिक Singleton एक ऐसी समस्या हल कर रहा था जो हमारे पास अब नहीं थी, जबकि एक टेस्टेबिलिटी समस्या पैदा कर रहा था जो हमारे पास थी। हमने इसे एक कंटेनर-प्रबंधित singleton बाइंडिंग में माइग्रेट किया, जहाँ भी ज़रूरत थी वहाँ इंजेक्ट किया, और वह अस्थिर टेस्ट लगभग 40 पंक्तियों के डिफेंसिव लॉकिंग कोड के साथ गायब हो गया जो असल में शुरू से कभी सही फ़िक्स था ही नहीं।"

Part 14 — Analogies & Architecture Fit

Analogies: - **A country's sitting president/PM** — the office structurally enforces "exactly one at a time"; nobody has to personally remember not to create a second one. Best single analogy for "the class enforces it, not the developer's discipline." - **A building's single reception desk** — every visitor is routed through the same one desk regardless of which entrance they used; the desk itself doesn't multiply just because more people show up. Captures the "single access point" half of the definition well, less so the lazy-creation half. - **A company's one official letterhead template** — every department must use the identical, centrally-controlled version rather than each keeping its own slightly-drifted copy. Useful for the config/consistency framing specifically. - **Weak analogy, worth naming as weak:** "a bank has one vault" — sounds right but breaks down immediately (real banks have branches, each with its own vault), so it accidentally illustrates *Multiton* (one per branch/key) better than true Singleton — worth having ready specifically to redirect an interviewer who offers this comparison.

उपमाएँ: - **एक देश का सत्तारूढ़ राष्ट्रपति/PM** — पद संरचनात्मक रूप से "एक समय में ठीक एक" लागू करता है; किसी को व्यक्तिगत रूप से यह याद रखने की ज़रूरत नहीं कि दूसरा न बनाए। "क्लास इसे लागू करती है, डेवलपर का अनुशासन नहीं" के लिए सबसे अच्छी अकेली उपमा। - **एक इमारत का एक अकेला रिसेप्शन डेस्क** — हर आगंतुक चाहे किसी भी प्रवेश द्वार से आया हो, उसी एक डेस्क से गुज़ारा जाता है; डेस्क खुद सिर्फ़ इसलिए नहीं बढ़ता कि ज़्यादा लोग आ गए। परिभाषा के "एकल एक्सेस बिंदु" वाले आधे हिस्से को अच्छी तरह पकड़ता है, आलसी-निर्माण वाले आधे को उतना नहीं। - **एक कंपनी का एक आधिकारिक लेटरहेड टेम्पलेट** — हर विभाग को केंद्रीय रूप से नियंत्रित समान संस्करण इस्तेमाल करना चाहिए, न कि अपनी थोड़ी-अलग कॉपी रखनी चाहिए। खास तौर पर config/संगति फ्रेमिंग के लिए उपयोगी। - **कमज़ोर उपमा, इसे कमज़ोर बताना ज़रूरी है:** "एक बैंक का एक तिजोरी है" — सही लगता है लेकिन तुरंत टूट जाता है (असली बैंकों की शाखाएँ होती हैं, हर एक की अपनी तिजोरी), इसलिए यह ग़लती से सच्चे Singleton से बेहतर *Multiton* को दर्शाता है।

Architecture fit: - **Clean/Hexagonal/Onion:** Singleton-as-global-access-point sits uncomfortably here — it's most defensible as an *infrastructure-layer* concern accessed via an interface the domain layer depends on, never as something the domain layer reaches for directly. - **DDD:** maps to nothing in the domain model itself — Aggregates, Entities, and Value Objects are all meant to be many-instanced. Singleton belongs, if anywhere, in the supporting infrastructure/application layers. - **Event-driven architecture:** a single event-bus client connection per service instance is a legitimate Singleton-shaped need — but the guarantee is process-scoped, so a genuinely fleet-wide guarantee needs idempotency keys or a distributed lock. - **CQRS:** no strong connection — stated plainly rather than forced. - **Cloud-native/Kubernetes:** worth one sentence, not more — a Singleton guarantees one instance *per pod/process*; it says nothing about the fleet.

आर्किटेक्चर फ़िट: - **Clean/Hexagonal/Onion:** Singleton-बतौर-वैश्विक-एक्सेस-बिंदु यहाँ असहज बैठता है — यह एक इन्फ्रास्ट्रक्चर-लेयर चिंता के तौर पर सबसे बचाव-योग्य है, एक इंटरफ़ेस के ज़रिए एक्सेस किया गया जिस पर डोमेन लेयर निर्भर करती है, कभी वह चीज़ नहीं जिसे डोमेन लेयर सीधे इस्तेमाल करे। - **DDD:** डोमेन मॉडल में खुद कुछ भी मैप नहीं होता — Aggregates, Entities, और Value Objects सभी कई-इंस्टेंस वाले होने चाहिए। Singleton, अगर कहीं भी, सहायक इन्फ्रास्ट्रक्चर/एप्लिकेशन लेयर्स में है। - **इवेंट-ड्रिवन आर्किटेक्चर:** प्रति सेवा इंस्टेंस एक अकेला इवेंट-बस क्लाइंट कनेक्शन एक वैध Singleton-आकार की ज़रूरत है — लेकिन गारंटी प्रोसेस-स्कोप्ड है, इसलिए एक सचमुच प्लूटि-व्यापी गारंटी को idempotency की या एक डिस्ट्रिब्यूटेड लॉक

चाहिए। - **CQRS**: कोई मज़बूत संबंध नहीं — खुलकर कहा गया, थोपा नहीं गया। - **Cloud-native/Kubernetes**: एक वाक्य लायक, ज़्यादा नहीं — एक Singleton प्रति-पॉड/प्रोसेस एक इंस्टेंस की गारंटी देता है; यह फ़्लीट के बारे में कुछ नहीं कहता।

✓ **Before you move on:** (1) Which analogy actually illustrates Multiton better than Singleton, and why is that worth catching? (2) In a Kubernetes deployment with 20 replica pods, how many Singleton instances of a given class actually exist across the fleet?

✓ **आगे बढ़ने से पहले:** (1) कौन-सी उपमा असल में Singleton से बेहतर Multiton को दर्शाती है, और यह पकड़ना क्यों ज़रूरी है? (2) 20 रेप्लिका पॉड्स वाले एक Kubernetes डिप्लॉयमेंट में, दी गई एक क्लास के कितने Singleton इंस्टेंसेज़ असल में पूरे फ़्लीट में मौजूद हैं?

Part 15 — SOLID, Performance & Concurrency

SOLID: the honest picture here is mostly negative, and saying so directly is a stronger interview answer than forcing a positive spin. **Single Responsibility** is often violated in practice, not by the pattern itself but by what accretes onto Singletons over time. **Open/Closed** has no strong connection either way. **Liskov Substitution** is not meaningfully engaged unless the Singleton implements an interface. **Interface Segregation** has no meaningful connection. **Dependency Inversion is the one Singleton actively fights** — DIP wants consumers to depend on abstractions provided to them, not reach out to a concrete global; a hard-coded `Logger::getInstance()` call site is a textbook DIP violation.

SOLID: यहाँ ईमानदार तस्वीर ज़्यादातर नकारात्मक है, और इसे सीधे कहना एक सकारात्मक मोड़ थोपने से बेहतर इंटरव्यू जवाब है। **Single Responsibility** व्यवहार में अक्सर उल्लंघित होता है, पैटर्न खुद से नहीं, बल्कि उस चीज़ से जो समय के साथ Singletons पर जमा होती जाती है। **Open/Closed** का किसी भी तरफ़ कोई मज़बूत संबंध नहीं। **Liskov Substitution** तभी सार्थक रूप से जुड़ता है जब Singleton एक इंटरफ़ेस इम्प्लीमेंट करता है। **Interface Segregation** का कोई सार्थक संबंध नहीं। **Dependency Inversion वह अकेला है जिससे Singleton सक्रिय रूप से लड़ता है** — DIP चाहता है कि कंज्यूमर्स उन्हें दिए गए ऐब्स्ट्रैक्शन्स पर निर्भर करें, एक कॉन्क्रीट ग्लोबल तक न पहुँचें; एक हार्ड-कोडेड `Logger::getInstance()` कॉल-साइट एक टेक्स्टबुक DIP उल्लंघन है।

Performance: the win is real but narrow — avoiding repeated expensive construction (file I/O, network handshake, parsing) by paying that cost once. It is not a general performance pattern and shouldn't be reached for on that basis alone.

परफ़ॉर्मेंस: फ़ायदा असली है लेकिन संकीर्ण — बार-बार महँगे निर्माण से बचना, वह लागत एक बार चुकाकर। यह एक सामान्य परफ़ॉर्मेंस पैटर्न नहीं है और सिर्फ़ इस आधार पर इसकी ओर नहीं पहुँचना चाहिए।

Concurrency: this is the section with the most genuine technical depth in this pattern, and where Part 2's Malaysia/Maybank data makes it non-optional. The unguarded `getInstance()` check-then-act sequence races under true concurrent execution. In **Java**, the historically correct fix is double-checked locking with the instance field marked `volatile` — verified via search against current sources: pre-JDK5, double-checked locking was actually broken due to instruction reordering allowing another thread to observe a partially-constructed object; JDK5+'s revised memory model, combined with `volatile`, fixes this. Simpler and now more

commonly recommended in Java: the enum-based singleton, where the JVM itself guarantees single, thread-safe instantiation with no manual locking at all. In **PHP**, the calculus is different by deployment model, exactly as detailed in Part 9. In **Go**, `sync.Once` is the idiomatic, already-correct primitive purpose-built for exactly this problem.

Concurrency: यह इस पैटर्न में सबसे असली तकनीकी गहराई वाला सेक्शन है, और जहाँ Part 2 का Malaysia/Maybank डेटा इसे वैकल्पिक नहीं बनाता। बिना गार्ड वाला `getInstance()` जाँच-फिर-कार्य क्रम असली समवर्ती निष्पादन के तहत दौड़ता है। **Java** में, ऐतिहासिक रूप से सही फ़िक्स `volatile` चिह्नित इंस्टेंस फ़ील्ड के साथ double-checked locking है — मौजूदा स्रोतों के खिलाफ़ खोजकर सत्यापित: JDK5 से पहले, double-checked locking असल में इंस्ट्रक्शन रीऑर्डिंग के कारण टूटी हुई थी, जो दूसरे थ्रेड को एक आंशिक रूप से बना ऑब्जेक्ट देखने देती थी; JDK5+ का संशोधित मेमोरी मॉडल, `volatile` के साथ मिलकर, इसे ठीक करता है। सरल और अब Java में ज़्यादा आमतौर पर सुझाया गया: enum-आधारित singleton, जहाँ JVM खुद बिना किसी मैनुअल लॉकिंग के सिंगल, थ्रेड-सेफ़ इंस्टैंशिएशन की गारंटी देता है। **PHP** में, गणना डिप्लॉयमेंट मॉडल के हिसाब से अलग है, ठीक जैसा Part 9 में बताया गया। **Go** में, `sync.Once` ठीक इसी समस्या के लिए बनाया गया मुहावरेदार, पहले से सही प्रिमिटिव है।

✓ **Before you move on:** (1) Which single SOLID principle does classic Singleton most directly fight, and why? (2) Why was double-checked locking actually broken in Java before JDK5, and what specific change fixed it?

✓ **आगे बढ़ने से पहले:** (1) क्लासिक Singleton सबसे सीधे किस अकेले SOLID सिद्धांत से लड़ता है, और क्यों? (2) JDK5 से पहले Java में double-checked locking असल में क्यों टूटी हुई थी, और किस खास बदलाव ने इसे ठीक किया?

Part 16 — Advantages, Disadvantages & Trade-offs

Dimension	Advantage	Disadvantage / trade-off
Performance	Avoids redundant expensive construction (I/O, parsing, connection setup)	Zero benefit for cheap-to-construct classes; pure overhead if misapplied
Scalability	Predictable, bounded resource usage (one pool, one connection)	Process-scoped only — provides no fleet-wide guarantee
Maintainability	One well-known place to find "the" instance of something	Accretes unrelated responsibilities over time
Readability	Simple, widely recognized pattern once named	Hidden dependency — reading a method's signature doesn't reveal it silently depends on global state
Security	Neutral	A shared mutable Singleton holding tenant- or user-specific state is an active data-leak risk
Testing	None inherent to the pattern itself	Actively adversarial to unit testing
Observability	One instance is a natural place to attach metrics	Harder to attribute behavior to a specific caller/request

हिंदी सार: परफॉर्मेंस — दोहराए गए महँगे निर्माण से बचाव, सस्ते-से-बनने वाली क्लासेज़ के लिए कोई फ़ायदा नहीं। स्केलेबिलिटी — पूर्वानुमेय, सीमित संसाधन इस्तेमाल, लेकिन सिर्फ़ प्रोसेस-स्कोप, कोई प्लैट-व्यापी गारंटी नहीं। मेटेनेबिलिटी — किसी चीज़ का "वह" इंस्टेंस ढूँढ़ने की एक जानी-पहचानी जगह, लेकिन समय के साथ असंबंधित ज़िम्मेदारियाँ जमा होती हैं। पठनीयता — एक बार नाम मिलने पर सादा, व्यापक रूप से पहचाना गया पैटर्न, लेकिन छिपी हुई डिपेंडेंसी। सुरक्षा — तटस्थ, लेकिन टेनेंट- या यूज़र-विशिष्ट स्थिति रखने वाला एक साझा परिवर्तनशील Singleton एक सक्रिय डेटा-लीक जोखिम है। टेस्टिंग — पैटर्न में खुद कोई अंतर्निहित फ़ायदा नहीं, यूनिट टेस्टिंग के सक्रिय रूप से विरोधी। ऑब्ज़र्वेबिलिटी — मेट्रिक्स जोड़ने के लिए एक स्वाभाविक जगह, लेकिन एक खास कॉलर/रिक्वेस्ट को व्यवहार बताना मुश्किल।

✓ **Before you move on:** (1) Name the one dimension where Singleton is close to a pure, low-risk win. (2) Name the dimension most responsible for mature teams preferring the DI-container alternative.

✓ **आगे बढ़ने से पहले:** (1) वह एक आयाम बताएँ जहाँ Singleton लगभग पूरी तरह फ़ायदेमंद, कम-जोखिम है। (2) वह आयाम बताएँ जो परिपक्व टीमों द्वारा DI-कंटेनर विकल्प को प्राथमिकता देने के लिए सबसे ज़्यादा ज़िम्मेदार है।

Part 17 — Pattern Comparisons

	Singleton	Static Class	Registry	Multiton	DI-Container Singleton Scope
Instances that exist	Exactly one (an object)	Zero (no instance at all)	Many, named	Many, keyed	Exactly one per binding (an object)
Can implement an interface / be polymorphic	Yes	No	Yes (each entry)	Yes (each entry)	Yes
Testable / mockable	Hard	Hard	Easier	Easier, per-key	Easy
Enforced by	The class itself	The language	Convention	Convention, per key	The container's configuration
Typical modern usage	Small, self-contained utilities	Pure stateless helper functions	Multiple named services/strategies	"One connection pool per shard"	Production services in mature codebases

हिंदी सार (तुलना टेबल): Singleton में ठीक एक इंस्टेंस (एक ऑब्जेक्ट) मौजूद है, क्लास खुद इसे लागू करती है, टेस्ट करना मुश्किल। Static Class में कोई इंस्टेंस नहीं, भाषा इसे लागू करती है, पॉलीमॉर्फिक नहीं हो सकती। Registry में कई नामित इंस्टेंसेज़ हैं, टेस्ट करना आसान है। Multiton में कई keyed इंस्टेंसेज़ हैं, प्रति-की आसान। DI-कंटेनर Singleton Scope में प्रति-बाइंडिंग ठीक एक इंस्टेंस है, कंटेनर के कॉन्फ़िगरेशन से लागू, टेस्ट करना आसान — यह परिपक्व कोडबेस में प्रोडक्शन सेवाओं का आम आधुनिक इस्तेमाल है।

Decision table:

Situation	Reach for
Exactly one instance, ever, self-contained utility, testability not a concern	Classic Singleton
Exactly one instance, but the class must stay unit-testable	DI-container singleton-scope binding
Several interchangeable stateless helper functions, no instance needed at all	Static class / free functions
Several named, independently-swappable instances (strategies, gateways)	Registry
"One per key" (one pool per shard, one client per tenant)	Multiton
Fleet-wide "exactly one," across multiple processes/pods	Neither — distributed lock / leader election

निर्णय टेबल — हिंदी सार: ठीक एक इंस्टेंस, हमेशा, स्वयं-निहित यूटिलिटी, टेस्टेबिलिटी चिंता नहीं → क्लासिक Singleton। ठीक एक इंस्टेंस, लेकिन क्लास यूनिट-टेस्ट करने-योग्य बनी रहनी चाहिए → DI-कंटेनर singleton-scope बाइंडिंग। कई अदल-बदल-योग्य स्टेटलेस हेल्पर फ़ंक्शन्स, कोई इंस्टेंस नहीं चाहिए → स्टैटिक क्लास/मुक्त फ़ंक्शन्स। कई नामित, स्वतंत्र रूप से बदली जा सकने वाली इंस्टेंसेज़ → Registry। "प्रति-की एक" → Multiton। पूरे फ़्लैट में "ठीक एक" → कोई नहीं — डिस्ट्रिब्यूटेड लॉक/लीडर इलेक्शन।

✓ **Before you move on:** (1) What's the one-sentence difference between Registry and Multiton? (2) Why does a DI-container singleton binding solve the same problem as classic Singleton without the same testability cost?

✓ **आगे बढ़ने से पहले:** (1) Registry और Multiton के बीच एक-वाक्य का अंतर क्या है? (2) एक DI-कंटेनर singleton बाइंडिंग उसी टेस्टेबिलिटी लागत के बिना क्लासिक Singleton जैसी ही समस्या क्यों हल करती है?

Part 18 — Production Bugs, AI-Generated Code Review & Testing

The flagship bug — the race in lazy check-then-act. Under any deployment model where multiple execution contexts can run concurrently within one process, an unguarded `getInstance()` can construct two separate instances if two callers both observe "not yet created" before either finishes constructing. Symptom: intermittent, hard-to-reproduce bugs where state that "should" be shared silently isn't. Debug by adding an instance-identity log (`spl_object_id($instance)`) at every call site during investigation.

मुख्य बग — lazy जाँच-फिर-कार्य में रেস। किसी भी डिप्लॉयमेंट मॉडल के तहत जहाँ एक प्रोसेस के भीतर कई निष्पादन संदर्भ समवर्ती रूप से चल सकते हैं, एक बिना-गार्ड वाला `getInstance()` दो अलग इंस्टेंसेज़ बना सकता है अगर दो कॉलर्स दोनों "अभी नहीं बना" देखें इससे पहले कि कोई बनाना पूरा करे। लक्षण: रुक-रुक कर, दोहराने में मुश्किल बग्स जहाँ वह स्थिति जो "साझा होनी चाहिए" चुपचाप नहीं है। जाँच के दौरान हर कॉल-साइट पर एक इंस्टेंस-पहचान लॉग (`spl_object_id($instance)`) जोड़कर डीबग करें।

Stale-state-across-tests bug. A classic Singleton's static state persists for the life of the PHP-FPM worker process or, worse, for the life of a test run if tests share a process — leading to

test pollution. Fix: either migrate to a container-managed instance the test harness can reset per test, or explicitly add a test-only reset hook.

टेस्ट्स के बीच बासी-स्थिति बग। एक क्लासिक Singleton की स्टैटिक स्थिति PHP-FPM वर्कर प्रोसेस के जीवन के लिए, या बदतर, टेस्ट रन के जीवन के लिए बनी रहती है अगर टेस्ट्स एक प्रोसेस साझा करते हैं — टेस्ट प्रदूषण की ओर ले जाते हुए। फ़िक्स: या तो एक कंटेनर-प्रबंधित इंस्टेंस में माइग्रेट करें जिसे टेस्ट हार्नेस प्रति-टेस्ट रीसेट कर सके या खुलकर एक टेस्ट-ओनली रीसेट हुक जोड़ें।

How AI coding assistants typically get this pattern wrong: - **Most common failure:** AI-generated Singleton implementations reliably include the private constructor and `getInstance()` but **frequently omit the `__clone()` and `__wakeup()` / `__unserialize()` guards** — the pattern "looks complete" while still allowing a second instance via cloning or deserialization. - **Second most common failure:** when asked to "make a thread-safe singleton," AI assistants frequently generate Java-style double-checked locking **translated literally into PHP**, including a nonexistent `volatile` keyword or a lock construct that doesn't map to PHP's actual concurrency model. - **What a reviewer should check before merging:** (1) are `__clone()` and `__wakeup()` / `__unserialize()` explicitly guarded; (2) does the chosen thread-safety mechanism actually correspond to this codebase's real concurrency model; (3) could a DI-container singleton binding deliver the same guarantee with better testability, and was that alternative actually considered.

AI कोडिंग असिस्टेंट्स आमतौर पर इस पैटर्न को कैसे ग़लत करते हैं: - **सबसे आम विफलता:** AI-जनित Singleton इम्प्लीमेंटेशन्स भरोसेमंद तरीके से प्राइवेट कंस्ट्रक्टर और `getInstance()` शामिल करती हैं लेकिन **अक्सर `__clone()` और `__wakeup()` / `__unserialize()` गार्ड्स छोड़ देती हैं** — पैटर्न "पूरा दिखता है" जबकि अब भी cloning या deserialization के ज़रिए दूसरा इंस्टेंस बनने देता है। - **दूसरी सबसे आम विफलता:** "एक थ्रेड-सेफ़ सिंगलटन बनाओ" पूछे जाने पर, AI असिस्टेंट्स अक्सर Java-शैली double-checked locking को **सीधे PHP में अनुवादित** करके जनरेट करते हैं, एक गैर-मौजूद `volatile` कीवर्ड सहित। - **मर्ज करने से पहले एक रिव्यूअर को क्या जाँचना चाहिए:** (1) क्या `__clone()` और `__wakeup()` / `__unserialize()` खुलकर गार्डेड हैं; (2) क्या चुना गया थ्रेड-सेफ़्टी तंत्र असल में इस कोडबेस के असली concurrency मॉडल से मेल खाता है; (3) क्या एक DI-कंटेनर singleton बाइंडिंग बेहतर टेस्टेबिलिटी के साथ वही गारंटी दे सकती थी।

Testing strategy — the identity/uniqueness test is the one category that matters most for this pattern:

टेस्टिंग रणनीति — पहचान/यूनीकनेस टेस्ट वह एक श्रेणी है जो इस पैटर्न के लिए सबसे ज़्यादा मायने रखती है:

```
public function test_get_instance_always_returns_the_same_object(): void
{
    $a = ConfigManager::getInstance();
    $b = ConfigManager::getInstance();

    $this->assertSame($a, $b); // identity, not just equal values
}

public function test_cloning_is_blocked(): void
{
    $this->expectException(\Error::class);
    $clone = clone ConfigManager::getInstance();
}
```

The critical detail, mirroring the equivalent rule from the Prototype handbook but inverted: here you assert `assertSame` (identity) to prove uniqueness is being *preserved*, and you explicitly test that cloning/unserialization is *blocked*.

महत्वपूर्ण विवरण, Prototype हैंडबुक के समकक्ष नियम को दर्शाता है मगर उलटा: यहाँ आप यूनिकनेस को संरक्षित साबित करने के लिए `assertSame` (पहचान) दावा करते हैं, और खुलकर टेस्ट करते हैं कि cloning/unserialization ब्लॉक है।

Code review checklist: constructor is private; `__clone()` throws or is private; `__wakeup()` / `__unserialize()` is guarded; an identity test (`assertSame`) exists, not just a type check; if "thread safety" is claimed, the mechanism is verified against this codebase's actual deployment/concurrency model; a DI-container alternative was genuinely considered.

कोड रिव्यू चेकलिस्ट: कंस्ट्रक्टर प्राइवेट है; `__clone()` थ्रो करता है या प्राइवेट है; `__wakeup()` / `__unserialize()` गार्डेड है; एक पहचान टेस्ट (`assertSame`) मौजूद है, सिर्फ़ एक टाइप-जाँच नहीं; अगर "थ्रेड सेफ़्टी" का दावा किया गया है, तो तंत्र इस कोडबेस के असली मॉडल के खिलाफ़ सत्यापित है; एक DI-कंटेनर विकल्प पर सचमुच विचार किया गया।

✓ **Before you move on:** (1) What's the most common AI-generated-code gap specifically in Singleton implementations? (2) Why must the uniqueness test use `assertSame` rather than checking the returned type alone?

✓ **आगे बढ़ने से पहले:** (1) Singleton इम्प्लीमेंटेशन्स में खास तौर पर सबसे आम AI-जनित-कोड गैप क्या है? (2) यूनिकनेस टेस्ट को सिर्फ़ लौटाई गई टाइप जाँचने के बजाय `assertSame` का इस्तेमाल क्यों करना चाहिए?

Part 19 — Refactoring Journey

Full code for every stage lives in `Singleton.php`; this narrates the reasoning connecting each one.

हर चरण का पूरा कोड `Singleton.php` में है; यह हर एक को जोड़ने वाला तर्क बताता है।

Stage 1 — Terrible (*where most engineers start, no shame in it*): `new ConfigManager()` scattered across a dozen files, each call re-parsing the same config from disk, no single source of truth.

चरण 1 — भयानक (जहाँ से ज़्यादातर इंजीनियर शुरू करते हैं, इसमें कोई शर्म नहीं): `new ConfigManager()` एक दर्जन फ़ाइलों में बिखरा, हर कॉल डिस्क से वही कॉन्फ़िग फिर से पार्स करती है, कोई एक सच्चाई का स्रोत नहीं।

Stage 2 — Bad, but a realistic first instinct (*often written by a mid-level engineer under time pressure*): a plain global variable or a static class property holding "the" config, set once somewhere in a bootstrap file and read everywhere else. Fixes the redundant-parsing problem but has none of Singleton's encapsulation.

चरण 2 — बुरा, मगर एक यथार्थवादी पहला अंतर्ज्ञान (अक्सर समय के दबाव में एक मिड-लेवल इंजीनियर द्वारा लिखा गया): एक सादा ग्लोबल वैरिएबल या एक स्टैटिक क्लास प्रॉपर्टी जो "वह" कॉन्फ़िग रखती है, कहीं एक बूटस्ट्रैप फ़ाइल में एक बार सेट होती है और बाक़ी हर जगह पढ़ी जाती है। दोहराए गए पार्सिंग की समस्या ठीक करता है लेकिन Singleton के एनकैप्सुलेशन में से कुछ भी नहीं है।

Stage 3 — Average, and the most dangerous stage in the whole journey (*a senior engineer moving fast, or code that later drifts as new call sites are added without matching*

review): a correctly-implemented classic Singleton — private constructor, static `getInstance()`, lazy init — but **missing the `__clone()` / `__wakeup()` guards**. Passes every normal test, looks finished, and silently allows a second instance the moment anything clones or unserializes it — exactly the AI-generated-code gap flagged in Part 18.

चरण 3 — औसत, और पूरी यात्रा का सबसे खतरनाक चरण (एक सीनियर इंजीनियर तेज़ी से आगे बढ़ रहा है, या कोड जो बाद में भटक जाता है): एक सही ढंग से इम्प्लीमेंट किया गया क्लासिक Singleton — प्राइवेट कंस्ट्रक्टर, स्टैटिक `getInstance()`, आलसी इनिट — लेकिन **`__clone()` / `__wakeup()` गार्ड्स गायब हैं**। हर सामान्य टेस्ट पास करता है, पूरा दिखता है, और जिस पल भी कुछ इसे क्लोन या अनसीरियालाइज़ करता है, चुपचाप दूसरा इंस्टेंस बनने देता है — ठीक Part 18 में चिह्नित AI-जनित-कोड गैप।

Stage 4 — Pattern correctly applied (*what a rigorous senior/staff engineer ships*): adds the clone/wakeup guards, adds the identity-uniqueness test proving them, and — critically for Malaysia-market prep per Part 2 — adds the concurrency guard appropriate to the actual deployment model.

चरण 4 — पैटर्न सही ढंग से लगाया गया (जो एक सख्त सीनियर/स्टाफ़ इंजीनियर शिप करता है): clone/wakeup गार्ड्स जोड़ता है, उन्हें साबित करने वाला पहचान-यूनीकनेस टेस्ट जोड़ता है, और — Part 2 के अनुसार Malaysia-बाज़ार तैयारी के लिए महत्वपूर्ण रूप से — असली डिप्लॉयमेंट मॉडल के अनुकूल concurrency गार्ड जोड़ता है।

Stage 5 — Production-ready (*staff-level judgment about the surrounding system, not just the class*): migrates from the classic static-accessor form to a DI-container singleton binding, preserving the "one instance" guarantee while restoring full testability.

चरण 5 — प्रोडक्शन-रेडी (आस-पास के सिस्टम के बारे में स्टाफ़-स्तर का निर्णय): क्लासिक स्टैटिक-एक्सेसर रूप से एक DI-कंटेनर singleton बाइंडिंग में माइग्रेट करता है, "एक इंस्टेंस" गारंटी को बनाए रखते हुए पूरी टेस्टेबिलिटी बहाल करता है।

✓ **Before you move on:** (1) Which stage is the most dangerous to leave in production, and why specifically that one rather than Stage 1 or 2? (2) What's the concrete difference between Stage 4 and Stage 5?

✓ **आगे बढ़ने से पहले:** (1) कौन-सा चरण प्रोडक्शन में छोड़ने के लिए सबसे खतरनाक है, और खास तौर पर वही क्यों? (2) चरण 4 और चरण 5 में ठोस अंतर क्या है?

Part 20 — Practices, Mistakes & Traps

Junior mistakes: forgetting the clone/wakeup guards entirely; making the constructor `public` "by accident" while still calling it a singleton; assuming `getInstance()` is automatically thread-safe in every language/runtime without checking.

शुरुआती गलतियाँ: clone/wakeup गार्ड्स को पूरी तरह भूल जाना; कंस्ट्रक्टर को "गलती से" `public` बनाना जबकि फिर भी इसे सिंगलटन कहना; बिना जाँचे यह मान लेना कि `getInstance()` हर भाषा/रनटाइम में अपने आप थ्रेड-सेफ़ है।

Mid-level mistakes: reaching for Singleton to solve "I don't want to pass this parameter through three function calls"; conflating "the DI container treats this as a singleton" with "I need to implement the GoF pattern by hand."

मिड-लेवल ग़लतियाँ: "मैं यह पैरामीटर तीन फ़ंक्शन कॉल्स से पास नहीं करना चाहता" हल करने के लिए Singleton की ओर पहुँचना; "DI कंटेनर इसे singleton मानता है" को "मुझे हाथ से GoF पैटर्न इम्प्लीमेंट करना है" से गड़मड़ करना।

Senior mistakes: translating a Java-style double-checked-locking solution literally into PHP without reasoning about whether PHP's actual concurrency model makes that mechanism meaningful; assuming a Singleton provides a fleet-wide uniqueness guarantee in a distributed/multi-pod deployment.

सीनियर ग़लतियाँ: बिना यह तर्क किए कि क्या PHP का असली concurrency मॉडल उस तंत्र को सार्थक बनाता है, एक Java-शैली double-checked-locking समाधान को सीधे PHP में अनुवादित करना; यह मान लेना कि एक Singleton एक डिस्ट्रिब्यूटेड/मल्टी-पॉड डिप्लॉयमेंट में फ़्लीट-व्यापी यूनिकनेस गारंटी देता है।

Interview follow-up questions that catch memorized-but-shallow understanding:

"You said this is thread-safe — thread-safe under what specific deployment model, and why does that qualifier matter in PHP specifically?" - "If I have 20 replica pods running this service, how many instances of your Singleton actually exist right now, across the whole fleet?" - "How would you unit-test a class that depends on this Singleton, without touching global state between tests?" - "Why did you choose a classic Singleton here instead of just registering this in the DI container as a singleton binding?"

इंटरव्यू फ़ॉलो-अप सवाल जो रटी-मगर-सतही समझ पकड़ते हैं: - "आपने कहा यह थ्रेड-सेफ़ है — किस खास डिप्लॉयमेंट मॉडल के तहत, और वह क्वालीफ़ायर PHP में खास तौर पर क्यों मायने रखता है?" - "अगर मेरे पास इस सेवा को चलाने वाले 20 रेप्लिका पॉड्स हैं, तो अभी पूरे फ़्लीट में आपके Singleton के असल में कितने इंस्टेंसेज़ मौजूद हैं?" - "आप इस Singleton पर निर्भर एक क्लास को कैसे यूनिट-टेस्ट करेंगे, टेस्ट्स के बीच वैश्विक स्थिति छुए बिना?" - "आपने यहाँ एक क्लासिक Singleton क्यों चुना, बजाय इसे DI कंटेनर में एक singleton बाइंडिंग के तौर पर रजिस्टर करने के?"

✓ **Before you move on:** (1) What's the difference between a mid-level and a senior mistake with this pattern, in one sentence? (2) Which interview follow-up specifically targets the process-scoped-vs-fleet-scoped misunderstanding?

✓ **आगे बढ़ने से पहले:** (1) इस पैटर्न के साथ एक मिड-लेवल और एक सीनियर ग़लती में एक-वाक्य का अंतर क्या है? (2) कौन-सा इंटरव्यू फ़ॉलो-अप खास तौर पर प्रोसेस-स्कोपेड-बनाम-फ़्लीट-स्कोपेड ग़लतफ़हमी को निशाना बनाता है?

Part 21 — Interview Question Bank & Coding Problems

Curated, high-signal, roughly 7 per level. Total questions delivered: 35.

चुनी हुई, उच्च-संकेत, प्रति स्तर लगभग 7। कुल दी गई सवालों की संख्या: 35।

Beginner (7)

1. *What problem does Singleton solve?* — Wrong: "it makes code run faster." — Good: "ensures only one instance of a class exists." — Excellent: adds "...and provides a single global access point to it." — Follow-up: "give a real example from a web app."

हिंदी: Singleton कौन-सी समस्या हल करता है? — ग़लत: "यह कोड को तेज़ चलाता है।" — अच्छा: "सुनिश्चित करता है कि एक क्लास का सिर्फ़ एक इंस्टेंस मौजूद हो।" — उत्कृष्ट: जोड़ता है "...और इसे एक अकेला वैश्विक एक्सेस बिंदु प्रदान करता है।" — फ़ॉलो-अप: "एक वेब ऐप से एक असली उदाहरण दीजिए।"

1. *How do you prevent a class from being instantiated with `new` from outside?* — Wrong: doesn't know constructors can have visibility modifiers. — Good: "make the constructor private." — Excellent: names that `protected` is used instead when controlled subclassing is intentionally supported. — Follow-up: "what happens if you forget this and leave it public?"

हिंदी: आप एक क्लास को बाहर से `new` द्वारा इंस्टैंशिएट होने से कैसे रोकते हैं? — ग़लत: नहीं जानता कि कंस्ट्रक्टर्स में विज़िबिलिटी मॉडिफ़ायर हो सकते हैं। — अच्छा: "कंस्ट्रक्टर को प्राइवेट बनाओ।" — उत्कृष्ट: बताता है कि `protected` का इस्तेमाल तब होता है जब जानबूझकर सबक्लासिंग सपोर्ट की जाती है। — फ़ॉलो-अप: "अगर आप इसे भूल जाएँ और पब्लिक छोड़ दें तो क्या होता है?"

1. *What does `getInstance()` typically do?* — Good: "returns the single instance, creating it the first time." — Excellent: explicitly separates the "does it exist yet" check from the "create and cache" step. — Follow-up: "is that creation eager or lazy in your example, and why?"

हिंदी: `getInstance()` आमतौर पर क्या करता है? — अच्छा: "अकेला इंस्टेंस लौटाता है, पहली बार इसे बनाते हुए।" — उत्कृष्ट: "क्या यह अभी मौजूद है" जाँच को "बनाओ और कैश करो" चरण से खुलकर अलग करता है। — फ़ॉलो-अप: "आपके उदाहरण में वह निर्माण eager है या lazy, और क्यों?"

1. *Name two real-world examples where Singleton fits well.* — Good: logger, config manager. — Excellent: adds *why* each fits. — Follow-up: "name one place people wrongly reach for it."

हिंदी: दो असली-दुनिया उदाहरण बताइए जहाँ Singleton अच्छी तरह फ़िट बैठता है। — अच्छा: logger, config manager। — उत्कृष्ट: हर एक क्यों फ़िट बैठता है, यह जोड़ता है। — फ़ॉलो-अप: "एक जगह बताइए जहाँ लोग ग़लत तरीक़े से इसकी ओर पहुँचते हैं।"

1. *What's the difference between Singleton and a static class?* — Wrong: "they're the same thing." — Good: "static class has no instance at all." — Excellent: adds that Singleton can implement interfaces and be polymorphic. — Follow-up: "when would that difference actually matter in real code?"

हिंदी: Singleton और एक स्टैटिक क्लास में क्या अंतर है? — ग़लत: "वे एक ही चीज़ हैं।" — अच्छा: "स्टैटिक क्लास का कोई इंस्टेंस बिल्कुल नहीं होता।" — उत्कृष्ट: जोड़ता है कि Singleton इंटरफ़ेसेज़ इम्प्लीमेंट कर सकता है और पॉलीमॉर्फिक हो सकता है। — फ़ॉलो-अप: "असली कोड में यह अंतर कब असल में मायने रखेगा?"

1. *Why might cloning a Singleton be a problem?* — Good: "it would create a second instance." — Excellent: names `__clone()` specifically and that it must be explicitly blocked. — Follow-up: "what about deserialization?"

हिंदी: एक Singleton को क्लोन करना समस्या क्यों हो सकती है? — अच्छा: "यह दूसरा इंस्टेंस बना देगा।" — उत्कृष्ट: खास तौर पर `__clone()` का नाम लेता है और कहता है कि इसे खुलकर ब्लॉक किया जाना चाहिए। — फ़ॉलो-अप: "deserialization के बारे में क्या?"

1. *Is Singleton a creational or structural pattern?* — Good: "creational." — Excellent: explains why. — Follow-up: "name the other four GoF creational patterns."

हिंदी: क्या Singleton क्रिएशनल या स्ट्रक्चरल पैटर्न है? — अच्छा: "क्रिएशनल।" — उत्कृष्ट: क्यों समझाता है। — फ़ॉलो-अप: "बाक़ी चार GoF क्रिएशनल पैटर्न्स के नाम बताइए।"

Intermediate (7)

1. *Walk through implementing a thread-safe Singleton in PHP.* — Wrong: translates Java double-checked locking literally. — Good: correctly notes PHP-FPM's per-request isolation means it's usually a non-issue. — Excellent: distinguishes PHP-FPM from Swoole/RoadRunner. — Follow-up: "what would you do differently if this ran on Swoole?"

हिंदी: PHP में एक थ्रेड-सेफ़ Singleton इम्प्लीमेंट करने से गुज़रें। — ग़लत: Java double-checked locking को सीधे अनुवादित करता है। — अच्छा: सही ढंग से नोट करता है कि PHP-FPM का प्रति-रिक्वेस्ट अलगाव मतलब यह आमतौर पर कोई मुद्दा नहीं है। — उत्कृष्ट: PHP-FPM को Swoole/RoadRunner से अलग करता है। — फ़ॉलो-अप: "अगर यह Swoole पर चले तो आप अलग क्या करेंगे?"

1. *Why is Singleton considered hard to unit test?* — Good: "the hard-coded `getInstance()` call resists mocking." — Excellent: proposes the concrete fix. — Follow-up: "show me how you'd refactor a consumer to make it testable."

हिंदी: Singleton को यूनिट-टेस्ट करना मुश्किल क्यों माना जाता है? — अच्छा: "हार्ड-कोडेड `getInstance()` कॉल मॉकिंग का विरोध करती है।" — उत्कृष्ट: ठोस फ़िक्स प्रस्तावित करता है। — फ़ॉलो-अप: "मुझे दिखाइए कि आप एक कंज़्यूमर को टेस्ट करने-योग्य बनाने के लिए कैसे रीफ़ैक्टर करेंगे।"

1. *What's the difference between Singleton and the Registry pattern?* — Good: "Registry holds many named instances; Singleton is exactly one." — Excellent: gives a concrete example needing Registry instead. — Follow-up: "when would you combine the two?"

हिंदी: Singleton और Registry पैटर्न में क्या अंतर है? — अच्छा: "Registry कई नामित इंस्टेंसेज़ रखती है; Singleton ठीक एक है।" — उत्कृष्ट: इसके बजाय Registry की ज़रूरत वाला एक ठोस उदाहरण देता है। — फ़ॉलो-अप: "आप दोनों को कब जोड़ेंगे?"

1. *Does a Singleton guarantee uniqueness across multiple servers/pods?* — Wrong: "yes." — Good: "no, only within one process." — Excellent: names what would actually be needed for fleet-wide uniqueness. — Follow-up: "what's a concrete scenario where this distinction caused a real bug?"

हिंदी: क्या एक Singleton कई सर्वर्स/पॉड्स में यूनिकनेस की गारंटी देता है? — ग़लत: "हाँ।" — अच्छा: "नहीं, सिर्फ़ एक प्रोसेस के अंदर।" — उत्कृष्ट: बताता है कि फ़्लैट-व्यापी यूनिकनेस के लिए असल में क्या चाहिए होगा। — फ़ॉलो-अप: "एक ठोस परिदृश्य क्या है जहाँ इस भेद ने एक असली बग पैदा की?"

1. *How would you make a Singleton class mockable in tests without abandoning the pattern entirely?* — Good: "extract an interface, inject it instead of calling `getInstance()` directly." — Excellent: describes a container-managed singleton binding as the natural landing point. — Follow-up: "is that still 'really' a GoF Singleton at that point?"

हिंदी: आप पैटर्न को पूरी तरह छोड़े बिना एक Singleton क्लास को टेस्ट्स में मॉक करने-योग्य कैसे बनाएँगे? — अच्छा: "एक इंटरफ़ेस निकालो, सीधे `getInstance()` कॉल करने के बजाय इसे इंजेक्ट करो।" — उत्कृष्ट: एक कंटेनर-प्रबंधित singleton बाइंडिंग को स्वाभाविक उतरने के बिंदु के तौर पर बताता है। — फ़ॉलो-अप: "क्या वह उस बिंदु पर अब भी 'सचमुच' एक GoF Singleton है?"

1. *What happens if two threads/coroutines call `getInstance()` at the exact same moment on an unguarded implementation?* — Good: "you might get two different instances." —

Excellent: explains the check-then-act race precisely. — Follow-up: "how would you reproduce this in a test?"

हिंदी: अगर दो थ्रेड्स/कोरूटीन्स एक बिना-गार्ड इम्प्लीमेंटेशन पर ठीक एक ही पल `getInstance()` कॉल करें तो क्या होता है? — अच्छा: "आपको दो अलग इंस्टेंसेज़ मिल सकती हैं।" — उत्कृष्ट: जाँच-फिर-कार्य रेस को सटीक रूप से समझाता है। — फॉलो-अप: "आप इसे एक टेस्ट में कैसे दोहराएँगे?"

1. *Why might eager initialization be preferable to lazy initialization in some Singleton implementations?* — Good: "avoids the concurrency race entirely." — Excellent: names the Swoole worker-boot-time pattern from Part 9. — Follow-up: "what's the downside of eager init if the resource is expensive and rarely used?"

हिंदी: कुछ Singleton इम्प्लीमेंटेशन्स में eager इनिशियलाइज़ेशन lazy इनिशियलाइज़ेशन से बेहतर क्यों हो सकता है? — अच्छा: "concurrency रेस को पूरी तरह टालता है।" — उत्कृष्ट: Part 9 का Swoole worker-boot-time पैटर्न नाम लेता है। — फॉलो-अप: "अगर संसाधन महँगा और शायद ही कभी इस्तेमाल होता है तो eager init का नुकसान क्या है?"

Senior (7)

1. *Design a `ConfigManager` that must be safely shared across a Swoole worker pool. Walk through your concurrency strategy.* — Excellent answer explains why PHP has no `volatile` equivalent and reasons about the actual memory model.

हिंदी: एक `ConfigManager` डिज़ाइन करें जिसे एक Swoole वर्कर पूल में सुरक्षित रूप से साझा किया जाना चाहिए। अपनी concurrency रणनीति से गुज़रें। — उत्कृष्ट जवाब बताता है क्यों PHP के पास कोई `volatile` समकक्ष नहीं है और असली मेमोरी मॉडल के बारे में तर्क करता है।

1. *Your team has 14 classes each implementing their own classic Singleton. What's your refactoring plan, and how do you sequence it safely?* — Excellent answer sequences it incrementally, keeping the old static accessor as a thin wrapper during transition.

हिंदी: आपकी टीम के पास 14 क्लासेज़ हैं जो हर एक अपना खुद का क्लासिक Singleton इम्प्लीमेंट करती हैं। आपकी रीफैक्टरिंग योजना क्या है, और आप इसे सुरक्षित रूप से कैसे क्रमबद्ध करते हैं? — उत्कृष्ट जवाब इसे क्रमिक रूप से क्रमबद्ध करता है, ट्रांज़िशन के दौरान पुराने स्टैटिक एक्सेसर को एक पतली रैपर के तौर पर रखते हुए।

1. *A Singleton-managed connection pool is showing degraded performance under high load. Where do you look first?* — Excellent answer also considers whether the Singleton itself has become a contention point.

हिंदी: एक Singleton-प्रबंधित कनेक्शन पूल उच्च लोड के तहत घटती परफॉर्मेंस दिखा रहा है। आप पहले कहाँ देखते हैं? — उत्कृष्ट जवाब यह भी विचार करता है कि क्या Singleton खुद एक विवाद बिंदु बन गया है।

1. *When would you explicitly choose NOT to use a Singleton for something that seems to want 'only one instance'?* — Excellent answer connects it back to Multiton or scoped DI bindings.

हिंदी: आप कब खुलकर किसी ऐसी चीज़ के लिए Singleton इस्तेमाल न करने का चुनाव करेंगे जो "सिर्फ़ एक इंस्टेंस" चाहती लगती है? — उत्कृष्ट जवाब इसे Multiton या स्कोपेड DI बाइंडिंग्स से जोड़ता है।

1. *Explain the DIP tension Singleton creates and how you'd defend using it anyway in a specific case.* — Excellent answer gives a genuinely defensible case.

हिंदी: Singleton जो DIP तनाव पैदा करता है उसे समझाएँ और आप एक खास मामले में फिर भी इसे इस्तेमाल करने का बचाव कैसे करेंगे। — उत्कृष्ट जवाब एक सचमुच बचाव-योग्य मामला देता है।

1. *How would you detect, in production, that a Singleton's uniqueness guarantee has been silently broken?* — Excellent answer proposes a lightweight runtime assertion/health-check.

हिंदी: आप प्रोडक्शन में कैसे पता लगाएँगे कि एक Singleton की यूनिकनेस गारंटी चुपचाप टूट गई है? — उत्कृष्ट जवाब एक हल्का रनटाइम असर्शन/हेल्थ-चेक प्रस्तावित करता है।

1. *Compare classic Singleton to Go's `sync.Once` and Java's enum-based singleton as concurrency-safety strategies.* — Excellent answer explains specifically *why* each is safer.

हिंदी: क्लासिक Singleton की तुलना Go के `sync.Once` और Java के enum-आधारित singleton से concurrency-सुरक्षा रणनीतियों के तौर पर करें। — उत्कृष्ट जवाब खास तौर पर बताता है क्यों हर एक ज़्यादा सुरक्षित है।

Staff/Principal (7)

1. *A Singleton-based feature-flag cache is causing a production incident: some pods are serving stale flags for up to 10 minutes after a flag change. Diagnose and fix.* — Excellent answer states this is a cache-invalidation problem wearing a Singleton costume, not a Singleton-correctness bug.

हिंदी: एक Singleton-आधारित feature-flag कैश एक प्रोडक्शन घटना का कारण बन रही है: कुछ पॉड्स एक फ़्लैग बदलाव के 10 मिनट बाद तक बासी फ़्लैग्स परोस रहे हैं। निदान करें और ठीक करें। — उत्कृष्ट जवाब कहता है कि यह Singleton का लबादा पहने एक कैश-इनवैलिडेशन समस्या है, कोई Singleton-सटीकता बग नहीं।

1. *Your org is migrating from PHP-FPM to a Swoole-based deployment for a service with a dozen existing classic Singletons. What's your migration risk assessment?* — Excellent answer identifies the "nothing changed but everything changed" migration trap.

हिंदी: आपका संगठन एक दर्जन मौजूदा क्लासिक Singletons वाली एक सेवा के लिए PHP-FPM से Swoole-आधारित डिप्लॉयमेंट में माइग्रेट कर रहा है। आपका माइग्रेशन जोखिम आकलन क्या है? — उत्कृष्ट जवाब "कुछ नहीं बदला मगर सब कुछ बदल गया" वाले माइग्रेशन जाल की पहचान करता है।

1. *Design an ADR recommending for or against classic Singleton usage as a team-wide standard, for a payments platform.* — Excellent answer lands on a defensible, non-absolutist position.

हिंदी: एक पेमेंट्स प्लेटफ़ॉर्म के लिए, टीम-व्यापी मानक के तौर पर क्लासिक Singleton इस्तेमाल के पक्ष या विपक्ष में एक ADR डिज़ाइन करें। — उत्कृष्ट जवाब एक बचाव-योग्य, ग़ैर-निरपेक्ष स्थिति पर उतरता है।

1. *How does Singleton interact with horizontal autoscaling, and what's the failure mode if an engineer assumes it doesn't?* — Excellent answer names the thundering-herd re-initialization spike.

हिंदी: Singleton क्षैतिज ऑटोस्केलिंग के साथ कैसे इंटरैक्ट करता है, और अगर एक इंजीनियर मान ले कि नहीं करता तो विफलता ढंग क्या है? — उत्कृष्ट जवाब थंडरिंग-हर्ड री-इनिशियलाइज़ेशन स्पाइक का नाम लेता है।

1. *A candidate on your team wants to implement a distributed cache using a Singleton. Coach them.* — Excellent answer redirects toward the actually-correct tools (Redis/Memcached).

हिंदी: आपकी टीम का एक उम्मीदवार एक Singleton का इस्तेमाल करके एक डिस्ट्रिब्यूटेड कैश इम्प्लीमेंट करना चाहता है। उसे कोच करें। — उत्कृष्ट जवाब असल में सही टूल्स (Redis/Memcached) की ओर पुनर्निर्देशित करता है।

1. *When, if ever, is it defensible to make Singleton state genuinely mutable and written from many call sites, versus read-only after initialization?* — Excellent answer distinguishes read-only-after-boot state from freely-mutable state.

हिंदी: कब, अगर कभी, Singleton स्थिति को कई कॉल-साइट्स से लिखी जाने वाली सचमुच परिवर्तनशील बनाना बचाव-योग्य है, बनाम इनिशियलाइज़ेशन के बाद रीड-ओनली? — उत्कृष्ट जवाब बूट-के-बाद-रीड-ओनली स्थिति को स्वतंत्र रूप से परिवर्तनशील स्थिति से अलग करता है।

1. *Retrospectively, what would you tell a team that over-used classic Singleton for years, without shaming the original decisions?* — Excellent answer frames the migration as a natural evolution.

हिंदी: पीछे मुड़कर देखें तो, आप उस टीम से क्या कहेंगे जिसने सालों तक क्लासिक Singleton का अति-इस्तेमाल किया, मूल फ़ैसलों को शर्मिदा किए बिना? — उत्कृष्ट जवाब माइग्रेशन को एक स्वाभाविक विकास के तौर पर फ़्रेम करता है।

Coding problems (solutions in `Singleton.php`): 1. Implement a `ConfigManager` singleton that lazily loads config from a JSON file exactly once, with `__clone()` / `__wakeup()` guards and a unit test proving instance identity across multiple `getInstance()` calls. 2. Implement a Swoole-aware `DbConnectionPool` singleton that initializes eagerly at worker boot (not lazily on first request) and demonstrate, via a commented-out lazy version, exactly which race the eager approach avoids.

कोडिंग समस्याएँ (हल `Singleton.php` में): 1. एक `ConfigManager` singleton इम्प्लीमेंट करें जो एक JSON फ़ाइल से कॉन्फ़िग को ठीक एक बार आलसी रूप से लोड करे, `__clone()` / `__wakeup()` गार्ड्स के साथ, और एक यूनिट टेस्ट जो कई `getInstance()` कॉल्स में इंस्टेंस पहचान साबित करे। 2. एक Swoole-जागरूक `DbConnectionPool` singleton इम्प्लीमेंट करें जो worker boot पर eager रूप से इनिशियलाइज़ हो, और एक कमेंट-आउट किए गए lazy वर्शन के ज़रिए दिखाएँ कि eager तरीक़ा ठीक-ठीक कौन-सी रेस से बचता है।

Total questions delivered: 35 (7 per level × 5 levels), plus 2 coding problems.

कुल दी गई सवालों की संख्या: 35 (7 प्रति स्तर × 5 स्तर), साथ ही 2 कोडिंग समस्याएँ।

APPENDIX

Part 22 — Learning Roadmap & Self-Assessment

Ranked resources: - *Beginner:* the GoF-pattern chapter on Singleton in any standard design-patterns reference; PHP manual pages on `__clone()` / `__wakeup()` / `__unserialize()` magic methods. - *Intermediate:* Laravel's official Service Container documentation (specifically the `bind()` vs `singleton()` distinction) — directly verified and cited in Part 11. - *Advanced:* Java Memory Model documentation/discussion on why pre-JDK5 double-checked locking was broken and what changed — directly verified and cited in Part 15; Go's `sync` package documentation on `sync.Once`.

दर्जाबद्ध संसाधन: - शुरुआती: किसी भी मानक डिज़ाइन-पैटर्न रेफरेंस में Singleton पर GoF-पैटर्न अध्याय; `__clone()` / `__wakeup()` / `__unserialize()` मैजिक मेथड्स पर PHP मैनुअल पेजेज़। - मध्यवर्ती: Laravel का आधिकारिक Service Container दस्तावेज़ीकरण (खास तौर पर `bind()` बनाम `singleton()` भेद) — Part 11 में सीधे सत्यापित और उद्धृत। - उन्नत: Java Memory Model दस्तावेज़ीकरण/चर्चा — Part 15 में सीधे सत्यापित और उद्धृत; Go के `sync` पैकेज का `sync.Once` पर दस्तावेज़ीकरण।

Self-Assessment — MCQs (answer key at the end):

स्वयं-मूल्यांकन — MCQs (अंत में उत्तर कुंजी):

1. What does `getInstance()` return on the *second* call in a correctly-implemented Singleton?
a) A new instance b) The same cached instance c) null d) An error

हिंदी: एक सही ढंग से इम्प्लीमेंट किए गए Singleton में दूसरी कॉल पर `getInstance()` क्या लौटाता है? a) एक नया इंस्टेंस b) वही कैशेड इंस्टेंस c) null d) एक एरर

1. Why must `__clone()` be guarded in a PHP Singleton? a) Performance b) It would silently create a second instance c) PHP requires it by default d) It's not actually necessary

हिंदी: PHP Singleton में `__clone()` को गार्ड क्यों किया जाना चाहिए? a) परफॉर्मेंस b) यह चुपचाप दूसरा इंस्टेंस बना देगा c) PHP को डिफ़ॉल्ट रूप से इसकी ज़रूरत है d) यह असल में ज़रूरी नहीं है

1. Under classic PHP-FPM, is an unguarded `getInstance()` vulnerable to a concurrency race?
a) Yes, always b) No — each request has its own isolated process/memory c) Only on Linux
d) Only with more than 2 CPU cores

हिंदी: क्लासिक PHP-FPM के तहत, क्या एक बिना-गार्ड `getInstance()` एक concurrency रेस के प्रति संवेदनशील है? a) हाँ, हमेशा b) नहीं — हर रिक्वेस्ट की अपनी अलग-थलग प्रोसेस/मेमोरी है c) सिर्फ़ Linux पर d) सिर्फ़ 2 से ज़्यादा CPU कोर के साथ

1. What does Laravel's `->singleton()` container binding do differently from `->bind()`? a) Nothing, they're identical b) `singleton()` caches and reuses the same instance after first resolution; `bind()` builds fresh every time c) `bind()` is faster d) `singleton()` requires a private constructor

हिंदी: Laravel की `->singleton()` कंटेनर बाइंडिंग `->bind()` से अलग क्या करती है? a) कुछ नहीं, वे एक जैसी हैं b) `singleton()` पहले रिजॉल्यूशन के बाद वही इंस्टेंस कैश और पुनः इस्तेमाल करता है; `bind()` हर बार ताज़ा बनाता है c) `bind()` तेज़ है d) `singleton()` को एक प्राइवेट कंस्ट्रक्टर चाहिए

1. What was broken about double-checked locking in Java before JDK5? a) It didn't compile b) Instruction reordering could expose a partially-constructed object to another thread c) It was too slow d) It required a private constructor

हिंदी: JDK5 से पहले Java में double-checked locking में क्या टूटा हुआ था? a) यह कंपाइल नहीं होता था b) इंस्ट्रक्शन रीऑर्डिंग दूसरे थ्रेड को एक आंशिक रूप से बना ऑब्जेक्ट दिखा सकती थी c) यह बहुत धीमा था d) इसे एक प्राइवेट कंस्ट्रक्टर चाहिए था

1. What does a Singleton guarantee in a 20-pod Kubernetes deployment? a) Exactly one instance across all 20 pods b) Exactly one instance per pod/process — up to 20 total c) Exactly zero instances d) It depends on the ingress controller

हिंदी: एक 20-पॉड Kubernetes डिप्लॉयमेंट में एक Singleton क्या गारंटी देता है? a) सभी 20 पॉड्स में ठीक एक इंस्टेंस b) प्रति-पॉड/प्रोसेस ठीक एक इंस्टेंस — कुल 20 तक c) ठीक शून्य इंस्टेंसेज़ d) यह ingress कंट्रोलर पर निर्भर करता है

1. What is Go's idiomatic primitive for exactly-once safe initialization? a) `volatile` b) `sync.Once` c) `getInstance()` d) A mutex is always required manually, no primitive exists

हिंदी: ठीक-एक-बार सुरक्षित इनिशियलाइज़ेशन के लिए Go का मुहावरेदार प्रिमिटिव क्या है? a) `volatile` b) `sync.Once` c) `getInstance()` d) हमेशा मैनुअल रूप से एक mutex चाहिए, कोई प्रिमिटिव मौजूद नहीं

Answer key: 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-b.

उत्तर कुंजी: 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-b.

Scenario questions: 1. Your team's `Logger::getInstance()` singleton is implicated in an intermittent test-flakiness issue in CI, where tests run in parallel worker processes. Diagnose likely causes and propose a fix. — Expected reasoning: check whether "parallel" here means separate OS processes or shared-process concurrency; propose either isolating test state per-process or migrating to a container-managed, test-resettable binding. 2. A staff engineer proposes replacing every classic Singleton in a legacy codebase with DI-container bindings in one large PR. Evaluate this plan. — Expected reasoning: flag the big-bang-rewrite risk; propose an incremental, dependency-ordered migration with tests added before each refactor.

परिदृश्य सवाल: 1. आपकी टीम का `Logger::getInstance()` सिंगलटन CI में एक रुक-रुक कर टेस्ट-अस्थिरता समस्या में शामिल है, जहाँ टेस्ट्स समानांतर वर्कर प्रोसेसेज़ में चलते हैं। संभावित कारणों का निदान करें और एक फ़िक्स प्रस्तावित करें। — अपेक्षित तर्क जाँचें कि यहाँ "समानांतर" का मतलब अलग OS प्रोसेसेज़ है या शेयर्ड-प्रोसेस concurrency; या तो प्रति-प्रोसेस टेस्ट स्थिति को अलग-थलग करने या एक कंटेनर-प्रबंधित, टेस्ट-रीसेट-योग्य बाइंडिंग में माइग्रेट करने का प्रस्ताव रखें। 2. एक स्टाफ़ इंजीनियर एक बड़े PR में एक विरासती कोडबेस में हर क्लासिक Singleton को DI-कंटेनर बाइंडिंग्स से बदलने का प्रस्ताव रखता है। इस योजना का मूल्यांकन करें। — अपेक्षित तर्क बड़े-धमाकेवाले फिर से लिखने के जोखिम को चिह्नित करें; हर रीफैक्टर से पहले टेस्ट्स जोड़े गए एक क्रमिक, डिपेंडेंसी-क्रमबद्ध माइग्रेशन का प्रस्ताव रखें।

One refactoring exercise: Take the Stage 3 (missing clone/wakeup guards) implementation from Part 19, add the guards, add the identity-uniqueness test from Part 18, and — if targeting

Malaysia specifically — add the Swoole-aware concurrency guard from Part 9, documenting which deployment model your guard assumes.

एक रीफैक्टिंग अभ्यास: Part 19 से चरण 3 (गायब clone/wakeup गार्ड्स) इम्प्लीमेंटेशन लें, गार्ड्स जोड़ें, Part 18 का पहचान-यूनीकनेस टेस्ट जोड़ें, और — अगर खास तौर पर Malaysia को लक्षित कर रहे हैं — Part 9 का Swoole-जागरूक concurrency गार्ड जोड़ें, यह दस्तावेज़ीकृत करते हुए कि आपका गार्ड किस डिप्लॉयमेंट मॉडल को मानता है।

One architecture/debugging scenario: A `FeatureFlagCache` Singleton is serving stale data for up to 10 minutes after flag changes across a fleet of autoscaled pods. Produce a short design note: root cause, why it's a cache-invalidation problem rather than a Singleton-correctness bug, and your proposed fix.

एक आर्किटेक्चर/डीबगिंग परिदृश्य: एक `FeatureFlagCache` Singleton ऑटोस्केल्ड पॉड्स के एक फ़्लीट में फ़्लैग बदलावों के 10 मिनट बाद तक बासी डेटा परोस रहा है। एक छोटा डिज़ाइन नोट बनाएँ: मूल कारण, यह क्यों Singleton-सटीकता बग के बजाय एक कैश-इनवैलिडेशन समस्या है, और आपका प्रस्तावित फ़िक्स।

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Singleton, Factory Method, और Builder तीनों क्रिएशनल पैटर्न हैं।
Lazy Initialization	आलसी (lazy) इनिशियलाइज़ेशन	<code>getInstance()</code> पहली कॉल पर आलसी रूप से इंस्टेंस बनाता है।
Check-then-act	जाँच-फिर-कार्य	<code>getInstance()</code> की जाँच-फिर-कार्य क्रम concurrency बग्स का स्रोत है।
Race Condition	रेस कंडीशन	दो कोरूटीन्स एक ही पल पर बनाने की कोशिश करें तो एक रेस कंडीशन पैदा होती है।
Double-Checked Locking	डबल-चेकड लॉकिंग	Java में double-checked locking को सही होने के लिए <code>volatile</code> चाहिए।
Dependency Inversion Principle (DIP)	डिपेंडेंसी इनवर्जन प्रिंसिपल	classic Singleton सबसे ज़्यादा DIP से लड़ता है।
Container-managed Singleton	कंटेनर-प्रबंधित सिंगलटन	Laravel की <code>->singleton()</code> बाइंडिंग एक कंटेनर-प्रबंधित सिंगलटन है।
Process-scoped	प्रोसेस-स्कोपड	एक Singleton की गारंटी प्रोसेस-स्कोपड है, फ़्लैट-स्कोपड नहीं।
Identity Test (<code>assertSame</code>)	पहचान टेस्ट	<code>assertSame(\$a, \$b)</code> यह साबित करता है कि दोनों कॉल्स एक ही ऑब्जेक्ट लौटाती हैं।
Memory Visibility	मेमोरी विज़िबिलिटी	PHP के पास Java जैसा कोई मेमोरी-विज़िबिलिटी प्रिमिटिव नहीं है।
Thundering Herd	थंडरिंग हर्ड	ऑटोस्केलिंग के दौरान कई पॉइंट्स का एक साथ कोल्ड-स्टार्ट करना एक थंडरिंग-हर्ड स्पाइक पैदा कर सकता है।
Leader Election	लीडर इलेक्शन	फ़्लैट-व्यापी यूनीकनेस के लिए लीडर इलेक्शन जैसा एक डिस्ट्रिब्यूटेड समन्वय तंत्र चाहिए।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Accretes (verb, "accrete")	जमा होना, बढ़ना	"Responsibilities accrete onto a shared object over time." समय के साथ ज़िम्मेदारियाँ एक साझा ऑब्जेक्ट पर जमा होती जाती हैं।
Uncomfortably	असहजता से	"The global-access idea sits uncomfortably inside a clean architecture." वैश्विक-एक्सेस विचार एक साफ़ आर्किटेक्चर के अंदर असहजता से बैठता है।
Stale (adjective)	बासी, पुराना	"The cache was serving stale data after the update." अपडेट के बाद कैश बासी डेटा परोस रहा था।
Blast radius	प्रभाव का दायरा	"A shared mutable object has a wide blast radius when it breaks." एक साझा परिवर्तनशील ऑब्जेक्ट के टूटने पर उसका प्रभाव दायरा बढ़ा होता है।
Papers over (idiom, "paper over")	ढाँकना, सतही तौर पर छिपाना	"A shorter TTL just papers over the real invalidation problem." एक छोटा TTL असली इनवैलिडेशन समस्या को सिर्फ़ ढाँकता है।
Genuinely	सचमुच, वास्तव में	"This is genuinely a testability problem, not a performance one." यह सचमुच एक टेस्टेबिलिटी समस्या है, परफॉर्मंस की नहीं।
Sidestep (verb)	टालना, बचकर निकलना	"Eager initialization sidesteps the race entirely." Eager इनिशियलाइज़ेशन रेस को पूरी तरह टाल देता है।
Retrospectively	पीछे मुड़कर देखने पर	"Retrospectively, the original decision made sense at the time." पीछे मुड़कर देखने पर, मूल फैसला उस वक़्त सही लगता था।
Blanket (adjective, "blanket answer")	सामान्य, हर हाल में लागू	"A blanket 'yes' answer misses the deployment-model nuance." एक सामान्य "हाँ" जवाब डिप्लॉयमेंट-मॉडल की बारीकी चूक जाता है।
Compounds (verb, "the cost compounds")	बढ़ता जाना, चक्रवृद्धि होना	"The wasted work compounds as order volume grows." ऑर्डर वॉल्यूम बढ़ने के साथ बर्बाद काम बढ़ता जाता है।
Defensible	बचाव-योग्य, तर्कसंगत	"A defensible position beats a blanket rule." एक बचाव-योग्य स्थिति एक सामान्य नियम से बेहतर है।
Auditable	जाँचने-योग्य	"A concrete decision rule makes the recommendation auditable." एक ठोस निर्णय नियम सिफ़ारिश को जाँचने-योग्य बनाता है।

Companion file: `Singleton.php` — *basic* → *clone/wakeup-guarded* → *Swoole-aware* → *real-world*
`ConfigManager/DbConnectionPool` progression, heavily commented, runnable with `php`
`Singleton.php`. Code file is English-only; this handbook is bilingual English + Hindi throughout.

साथी फ़ाइल: `Singleton.php` — *basic* → *clone/wakeup-guarded* → *Swoole-aware* → असली-दुनिया
`ConfigManager/DbConnectionPool` क्रम, भारी टिप्पणियों के साथ, `php Singleton.php` से रनेबल। कोड फ़ाइल सिर्फ़
अंग्रेज़ी में है; यह हैडबुक पूरी तरह अंग्रेज़ी + हिंदी द्विभाषी है।